

Java Version: 22

Year: 2024

Feature: Region Pinning for G1

UniqueClassName: RegionPinningG1

Definition

Region Pinning for G1 is an improvement in the **G1 (Garbage-First) Garbage Collector** introduced in **Java 22**.

It allows certain **G1 heap regions to be temporarily pinned**, meaning that G1 can prevent those regions from being relocated during garbage collection while they are being used by operations that require stable object locations.

This is particularly important for **JNI critical regions**, where native code may temporarily access Java heap memory directly.

Why Region Pinning?

Before this improvement, operations that pinned heap memory could interfere with G1's normal evacuation process. Region pinning allows G1 to handle these situations more efficiently.

Key idea:

Normal G1:

Object → Region A



GC evacuation



Object → Region B

With Region Pinning:

Object → Region A



Region pinned

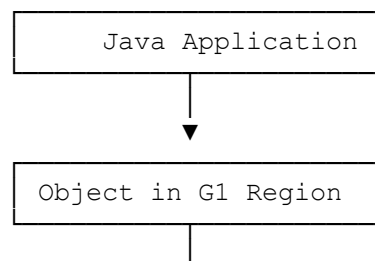


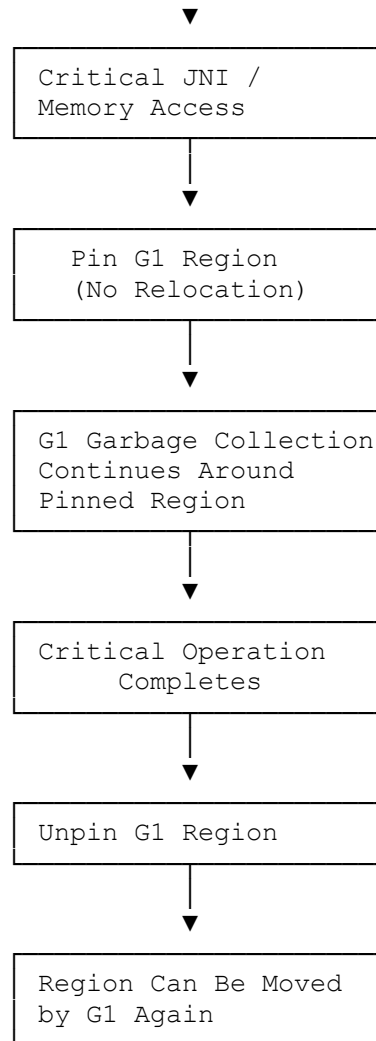
Native/JNI operation completes



Region becomes movable again

Flowchart





Java Code Example

Region pinning is primarily a **JVM/G1 implementation feature**, so there is **no ordinary Java API such as `RegionPinningG1.pin()`** that application code calls directly.

A Java application can nevertheless use JNI/native code that performs critical access:

```
public class RegionPinningG1 {  
  
    public static void main(String[] args) {  
        System.out.println("G1 Region Pinning Example");  
  
        byte[] data = new byte[1024 * 1024];  
  
        for (int i = 0; i < data.length; i++) {  
            data[i] = (byte) (i % 256);  
        }  
  
        System.out.println("Array allocated: " + data.length + " bytes");  
    }  
}
```

```

/*
 * In applications using JNI critical access,
 * the JVM/G1 collector can pin relevant regions
 * while native code accesses heap memory.
 */

System.gc();

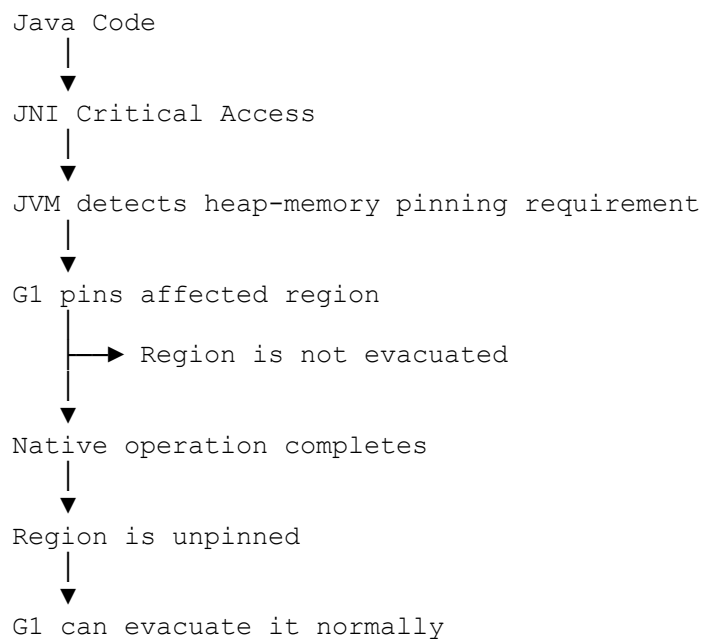
System.out.println("Application continues...");
}
}

```

Important Point

The above Java program **does not explicitly pin a G1 region**. The JVM performs region pinning internally when the relevant native/JNI mechanisms require it.

Conceptually:



Benefits

Benefit	Description
Better JNI support	Helps G1 handle critical native-memory access
Avoids unsafe relocation	Objects in pinned regions aren't moved while they must remain stable

Less GC disruption	G1 can continue collecting other regions
Improved interoperability	Better interaction between Java heap and native code
Automatic management	JVM handles pinning/unpinning internally

In one sentence:

Java 22's G1 Region Pinning allows G1 to temporarily keep selected heap regions from being relocated during operations that require stable heap addresses, especially critical JNI access.

Java 22 — Statements Before super(...) (Preview)

Java Version: 22

Year: 2024

Feature: Statements before super(...) (Preview)

UniqueClassName: StatementsBeforeSuper

Definition

Statements before super(...) is a **preview feature introduced in Java 22** that allows certain statements to appear **before an explicit constructor invocation** such as `super(...)` or `this(...)`.

Traditionally, a constructor must begin with `super(...)` or `this(...)` when an explicit constructor invocation is used. Java 22 relaxed this restriction so that **statements that do not access the object being constructed** can execute before the superclass constructor.

This is useful when constructor arguments need to be **validated, transformed, or calculated** before calling `super(...)`.

Traditional Java

```
class Employee extends Person {

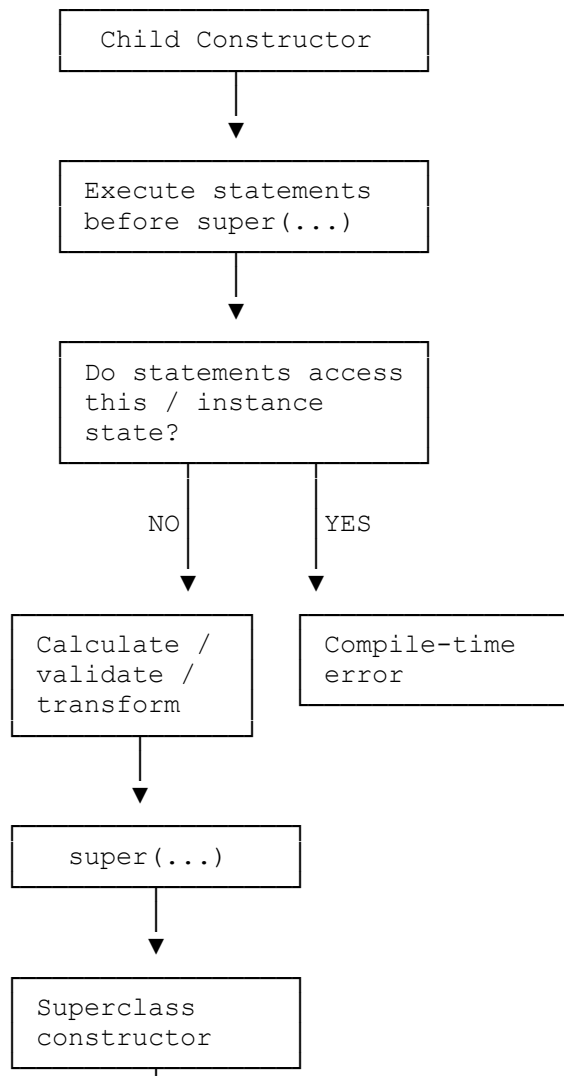
    Employee(String name) {
        super(name.toUpperCase()); // must come first
    }
}
```

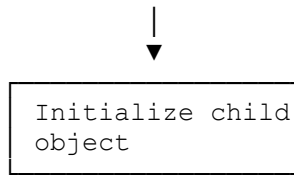
Java 22 Preview

```
class Employee extends Person {  
    Employee(String name) {  
        String normalizedName = name.trim().toUpperCase();  
        super(normalizedName);  
    }  
}
```

The important restriction is that the statements before `super(...)` **cannot use this or access instance state**, because the superclass constructor has not run yet.

Flowchart





Java Code Example

Because this is a **preview feature in Java 22**, compile and run it with `--enable-preview`.

```
class Person {

    private final String name;

    Person(String name) {
        this.name = name;
        System.out.println("Person constructor: " + name);
    }
}

public class StatementsBeforeSuper extends Person {

    private final int age;

    StatementsBeforeSuper(String name, int age) {

        // Statement before super(...)
        String normalizedName = name.trim();

        // Another statement before super(...)
        if (normalizedName.isEmpty()) {
            throw new IllegalArgumentException(
                "Name cannot be empty"
            );
        }

        // super(...) is called after the statements above
        super(normalizedName);

        // Child-class initialization
        this.age = age;
    }

    public void display() {
        System.out.println("Name: " + getName());
        System.out.println("Age: " + age);
    }

    private String getName() {
        return "Employee";
    }

    public static void main(String[] args) {
```

```

        StatementsBeforeSuper employee =
            new StatementsBeforeSuper("  Anurag  ", 30);

        employee.display();
    }
}

```

Simplified Example

The main idea can be represented as:

```

class Child extends Parent {

    Child(String value) {

        // Allowed: independent calculation
        String processed = value.trim();

        // Allowed: validation
        if (processed.isEmpty()) {
            throw new IllegalArgumentException();
        }

        // Explicit constructor invocation
        super(processed);
    }
}

```

What Is Allowed?

Statements before super(...) can perform work such as:

```

Child(String input) {

    String value = input.trim();

    int length = value.length();

    if (length == 0) {
        throw new IllegalArgumentException();
    }

    super(value);
}

```

What Is Not Allowed?

You cannot use the incompletely constructed object:

```

Child(String input) {

```

```
this.someMethod(); // ✗ Not allowed

super(input);
}
```

Likewise, accessing an instance field before the superclass constructor has completed is prohibited:

```
Child(String input) {

    System.out.println(this.value); // ✗ Not allowed

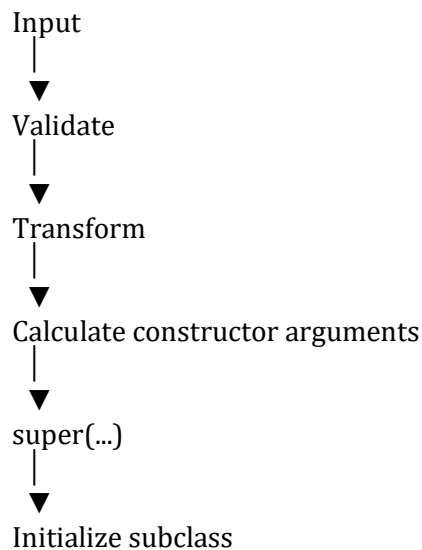
    super(input);
}
```

Why Was This Feature Introduced?

Before Java 22, developers often had to move calculations into:

- static helper methods,
- separate factory methods,
- superclass constructor arguments containing complicated expressions, or
- additional constructors.

Java 22 preview makes constructor code more natural:



Key Point

Statements before `super(...)` (Preview) in Java 22 allow safe, non-instance-dependent statements to execute before a superclass constructor invocation, making it easier to prepare constructor arguments and perform validation before object construction proceeds.

Java 22 — Foreign Function & Memory API

Java Version: 22

Year: 2024

Feature: Foreign Function & Memory API (Final)

UniqueClassName: ForeignFunctionMemoryAPI

Definition

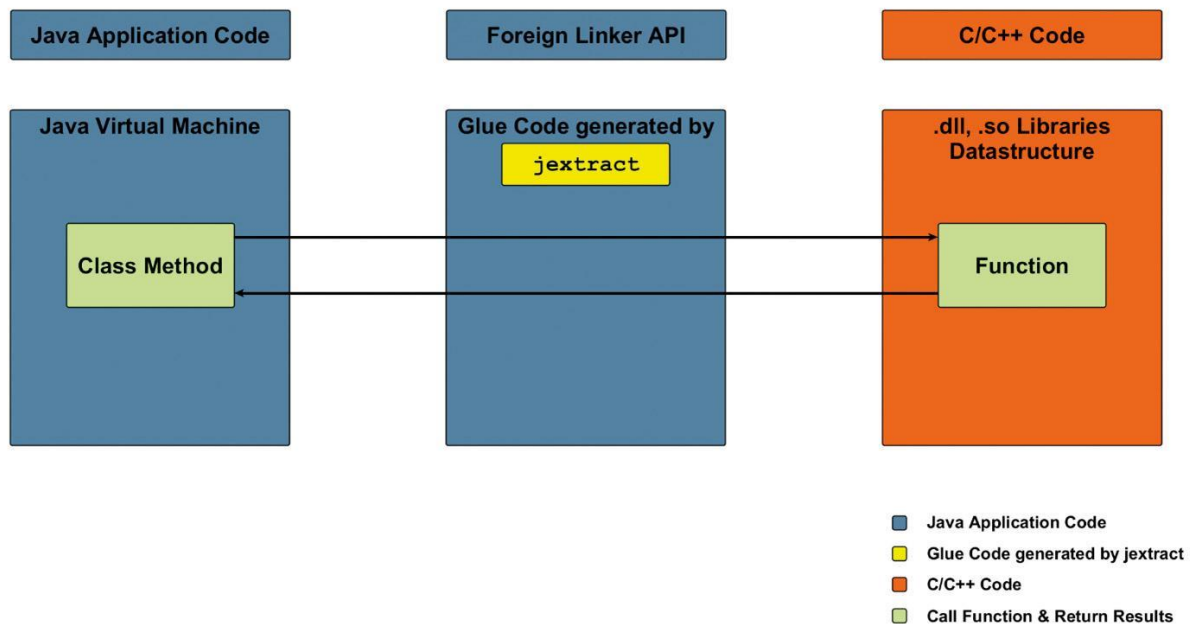
The **Foreign Function & Memory (FFM) API** became a **final feature** in **Java 22** through **JEP 454**.

It provides a standard Java API for:

1. **Calling native/foreign functions** outside the JVM, such as C library functions.
2. **Accessing foreign memory** outside the Java heap.
3. Managing native memory with explicit **scopes/arenas** rather than relying on unsafe low-level mechanisms.

The API primarily uses:

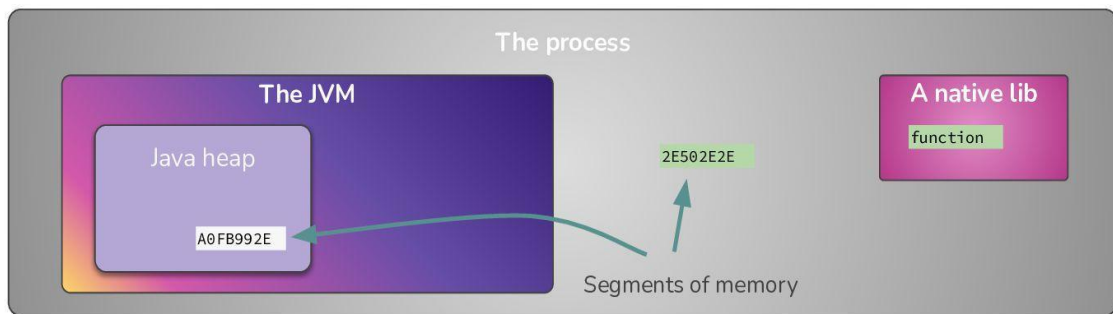
- Linker — connects Java code to foreign functions.
- SymbolLookup — finds native functions.
- MethodHandle — invokes the foreign function.
- MemorySegment — represents memory.
- Arena — controls the lifetime of allocated foreign memory.
- ValueLayout — describes the memory data type/layout.



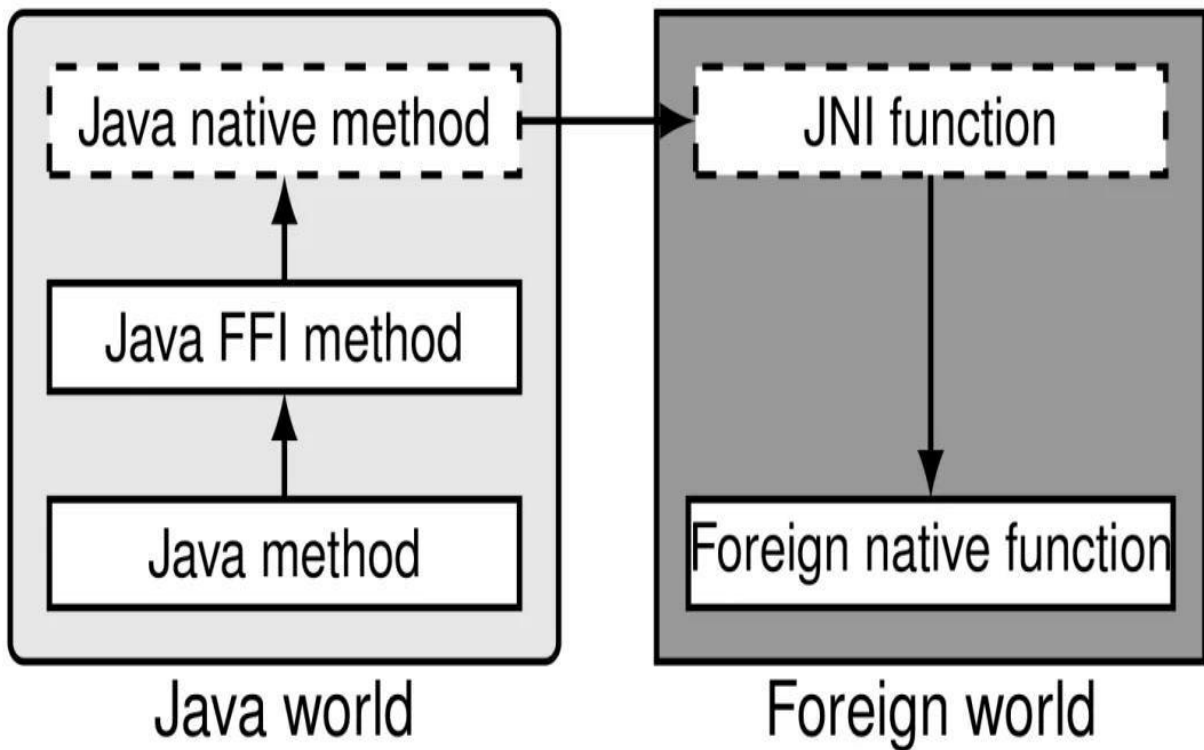


MemorySegment

Represents a contiguous "segment" of memory



The Over simplified "chart"™



```

import java.lang.foreign.*;
import static java.lang.foreign.ValueLayout.*;

public class PanamaDemo {
    public static void main(String[] args) throws Throwable {
        try (var linker = Linker.nativeLinker();
             var lib = SymbolLookup.loaderLookup();
             var arena = Arena.ofConfined()) {

            var printf = linker.downcallHandle(
                lib.find("printf").get(),
                FunctionDescriptor.of(JAVA_INT, ADDRESS, JAVA_INT));

            String msg = "Hello from native printf - %d + %d = %d\n";
            var str = arena.allocateFrom(msg);

            try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
                for (int i = 0; i < 50_000; i++) {
                    int x = i;
                    exec.submit(() -> printf.invoke(str, x, x*x, x + x*x));
                }
            }

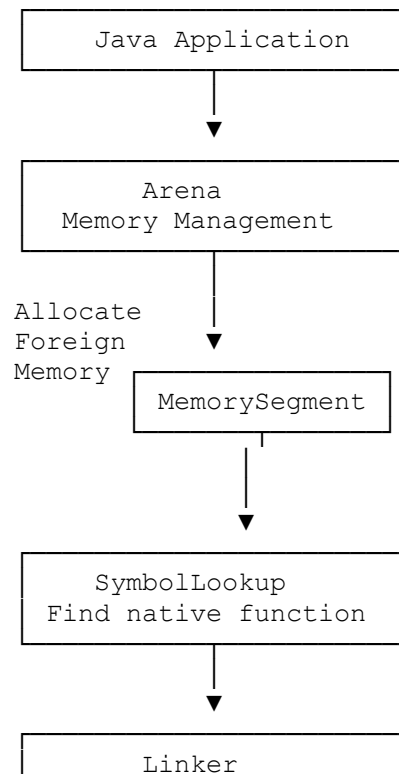
            System.out.println("50,000 native calls done - all from pure Java!");
        }
    }
}

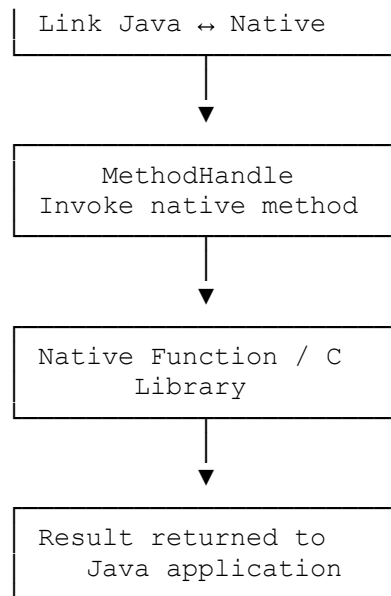
```

Load libc and call printf()
directly – no JNI!

Call real C printf 50,000 times
using virtual threads

Flowchart





Java Code Example — Calling a Native C Function

A simple example is calling the C standard-library `strlen()` function from Java.

```
import java.lang.foreign.Arena;
import java.lang.foreign.FunctionDescriptor;
import java.lang.foreign.Linker;
import java.lang.foreign.MemorySegment;
import java.lang.foreign.ValueLayout;
import java.lang.invoke.MethodHandle;

public class ForeignFunctionMemoryAPI {

    public static void main(String[] args) throws Throwable {

        // Create an Arena for managing foreign memory
        try (Arena arena = Arena.ofConfined()) {

            // Allocate a native string
            MemorySegment string =
                arena.allocateUtf8String("Hello Java FFM");

            // Get the system linker
            Linker linker = Linker.nativeLinker();

            // Look up strlen from the standard C library
            MemorySegment strlenAddress =
                linker.defaultLookup()
                    .find("strlen")
                    .orElseThrow();

            // Describe the native function:
```

```

        // size_t strlen(const char*)
        FunctionDescriptor strlenDescriptor =
            FunctionDescriptor.of(
                ValueLayout.JAVA_LONG,
                ValueLayout.ADDRESS
            );

        // Create a MethodHandle for strlen
        MethodHandle strlen =
            linker.downcallHandle(
                strlenAddress,
                strlenDescriptor
            );

        // Call the native C function
        long length =
            (long) strlen.invokeExact(string);

        System.out.println("String length = " + length);
    }
}

```

Expected Output

String length = 14

The string:

Hello Java FFM

contains 14 characters.

How the Code Works

1. Create an Arena

```
try (Arena arena = Arena.ofConfined()) {
```

An Arena manages the lifetime of foreign memory.

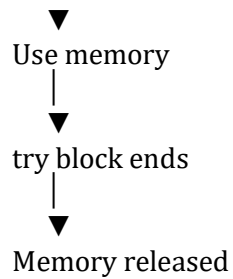
When the try block finishes, the allocated native memory is automatically released.

Arena created



Allocate native memory





2. Allocate Foreign Memory

```
MemorySegment string =  
    arena.allocateUtf8String("Hello Java FFM");
```

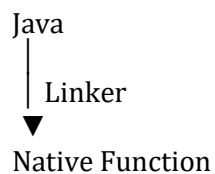
This creates a native-memory representation of the Java string.

MemorySegment provides a safe abstraction for accessing foreign memory.

3. Obtain the Native Linker

```
Linker linker = Linker.nativeLinker();
```

The Linker connects Java code with native functions.

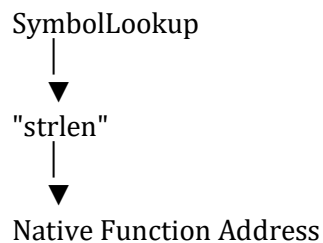


4. Find the Native Function

```
MemorySegment strlenAddress =  
    linker.defaultLookup()  
        .find("strlen")  
        .orElseThrow();
```

This searches for the native strlen function.

Conceptually:



5. Describe the Function

```
FunctionDescriptor strlenDescriptor =  
    FunctionDescriptor.of(  
        ValueLayout.JAVA_LONG,  
        ValueLayout.ADDRESS  
    );
```

This describes:

`size_t strlen(const char*);`

as:

Return type → `JAVA_LONG`
Argument → `ADDRESS`

6. Create a Downcall Handle

```
MethodHandle strlen =  
    linker.downcallHandle(  
        strlenAddress,  
        strlenDescriptor  
    );
```

A **downcall** means:

Java
|
▼
Native function

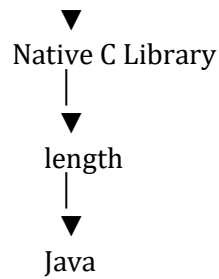
The opposite direction—native code calling Java—is called an **upcall**.

7. Invoke the Native Function

```
long length =  
    (long) strlen.invokeExact(string);
```

The Java program invokes the native C function directly.

Java MemorySegment
|
▼
MethodHandle
|
▼
strlen()
|



Major Components

Component	Purpose
Arena	Controls lifetime of foreign memory
MemorySegment	Represents foreign/native memory
Linker	Links Java with native functions
SymbolLookup	Locates native symbols/functions
MethodHandle	Invokes linked native functions
FunctionDescriptor	Describes native function signature
ValueLayout	Describes memory/value layouts

FFM vs JNI

JNI	Foreign Function & Memory API
Requires native glue code	Can call native functions directly
More boilerplate	More declarative Java API
Native compilation often required	No JNI wrapper required for many use cases
More difficult memory management	Arena and MemorySegment provide structured management
Older interoperability mechanism	Modern Java foreign-memory/function mechanism

Key Point

Java 22's Foreign Function & Memory API provides a safer, structured way for Java applications to call native functions and access foreign memory without the traditional JNI boilerplate, using APIs such as Linker, MemorySegment, Arena, and MethodHandle.

Java 22 — Unnamed Variables & Patterns

Java Version: 22

Year: 2024

Feature: Unnamed Variables & Patterns (Final)

UniqueClassName: UnnamedVariablesPatterns

Definition

Unnamed Variables & Patterns became a **final feature** in **Java 22** through **JEP 456**.

It allows developers to use the underscore (`_`) to indicate that a variable or pattern is **intentionally unused**.

Instead of inventing a variable name that will never be used, you can write `_`.

This is especially useful with:

- Local variables
- catch parameters
- Lambda parameters
- Pattern matching
- switch pattern matching

Traditional Code

```
try {
    Integer.parseInt("ABC");
} catch (NumberFormatException exception) {
    System.out.println("Invalid number");
}
```

If the exception object isn't needed, Java 22 allows:

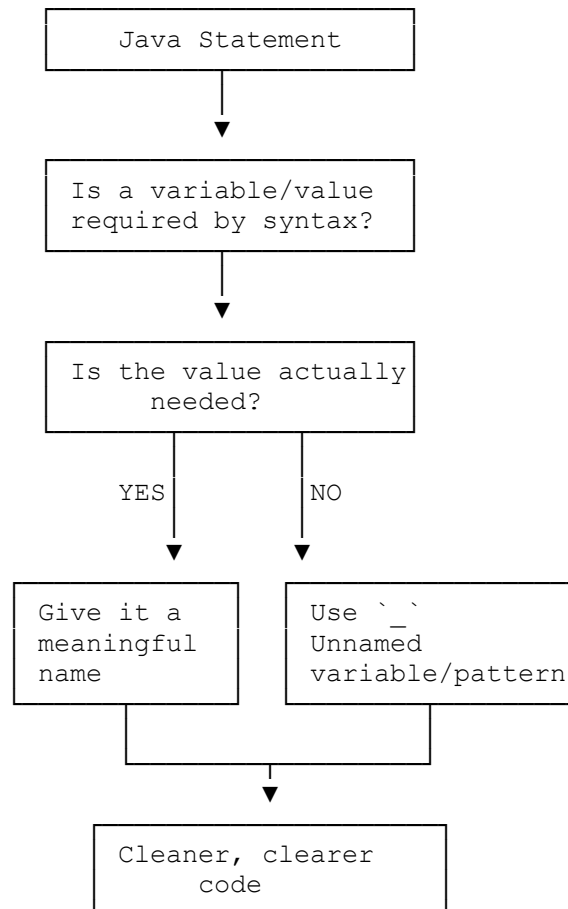
```
try {
    Integer.parseInt("ABC");
} catch (NumberFormatException _) {
    System.out.println("Invalid number");
}
```

}

The `_` clearly communicates:

"I intentionally don't need this value."

Flowchart



Java Code Example

```
public class UnnamedVariablesPatterns {  
    public static void main(String[] args) {  
        // Unnamed variable in a lambda  
        java.util.List<String> names =  
            java.util.List.of("Anurag", "Rahul", "Priya");  
    }  
}
```

```

        names.forEach(_ -> System.out.println("Hello"));

        // Unnamed variable in a catch block
        try {
            Integer.parseInt("ABC");
        } catch (NumberFormatException _) {
            System.out.println("Invalid number");
        }
    }
}

```

Expected Output

```

Hello
Hello
Hello
Invalid number

```

Unnamed Pattern Example

Unnamed patterns are particularly useful with **pattern matching**.

```

public class UnnamedVariablesPatterns {

    public static void main(String[] args) {

        Object value = "Java";

        if (value instanceof String _) {
            System.out.println("The value is a String");
        }
    }
}

```

Here:

String _

means:

The object must be a String, but I don't need to give the matched value a name.

Without an unnamed pattern, you might write:

```

if (value instanceof String text) {
    System.out.println("The value is a String");
}

```

But text is never used.

With Java 22:

```
if (value instanceof String _) {  
    System.out.println("The value is a String");  
}
```

is clearer.

Unnamed Variables in switch

Unnamed patterns are also useful in switch:

```
public class UnnamedVariablesPatterns {  
    static String describe(Object value) {  
        return switch (value) {  
            case String _ ->  
                "It is a String";  
            case Integer _ ->  
                "It is an Integer";  
            case null ->  
                "It is null";  
            default ->  
                "Unknown type";  
        };  
    }  
  
    public static void main(String[] args) {  
        System.out.println(describe("Java"));  
        System.out.println(describe(100));  
        System.out.println(describe(10.5));  
        System.out.println(describe(null));  
    }  
}
```

Output

```
It is a String  
It is an Integer  
Unknown type  
It is null
```

Unnamed Lambda Parameters

Suppose a lambda requires two parameters but only one is useful.

```
java.util.Map<String, Integer> scores =
    java.util.Map.of(
        "Java", 95,
        "Python", 90
    );

scores.forEach((name, _) ->
    System.out.println(name)
);
```

Here:

(name, _)

means:

name → used
_ → intentionally ignored

Important Rules

1. _ cannot be used as an ordinary variable name

This is invalid:

```
int _ = 10; // ✗ Compile-time error
```

Java reserves _ for unnamed variables and patterns.

2. The unnamed variable cannot be referenced

```
catch (Exception _) {
    System.out.println(_); // ✗ Not allowed
}
```

The whole purpose is that the value is **not accessed**.

3. Multiple _ variables are allowed

For example:

```
var list = java.util.List.of("A", "B", "C");
```

```
list.forEach((_ , _) ->
    System.out.println("Item")
);
```

Each `_` represents an independently unnamed value.

Before vs Java 22

Before

```
list.forEach(item -> {
    System.out.println("Processing");
});
```

`item` isn't actually used.

Java 22

```
list.forEach(_ -> {
    System.out.println("Processing");
});
```

The intent is immediately obvious.

Benefits

Benefit	Description
Less boilerplate	No meaningless variable names
Clear intent	<code>_</code> explicitly means "not used"
Cleaner patterns	Useful with <code>instanceof</code> and <code>switch</code>
Better readability	Focuses attention on values that matter
Avoids misleading names	Prevents unused names such as <code>ignored</code> , <code>unused</code> , or <code>temp</code>

Key Point

Unnamed Variables & Patterns in Java 22 allow `_` to represent intentionally unused variables and pattern components, making lambda expressions, exception handlers, and pattern matching cleaner and more expressive.

Java 22 — Class-File API (Preview)

Java Version: 22

Year: 2024

Feature: Class-File API (Preview)

UniqueClassName: ClassFileAPI

Definition

The **Class-File API** is a **preview feature introduced in Java 22** through **JEP 457**.

It provides a standard Java API for **parsing, generating, and transforming Java class files (.class files)**.

A class file contains the JVM bytecode and metadata produced by the Java compiler. Previously, developers commonly used third-party bytecode libraries such as ASM for manipulating class files.

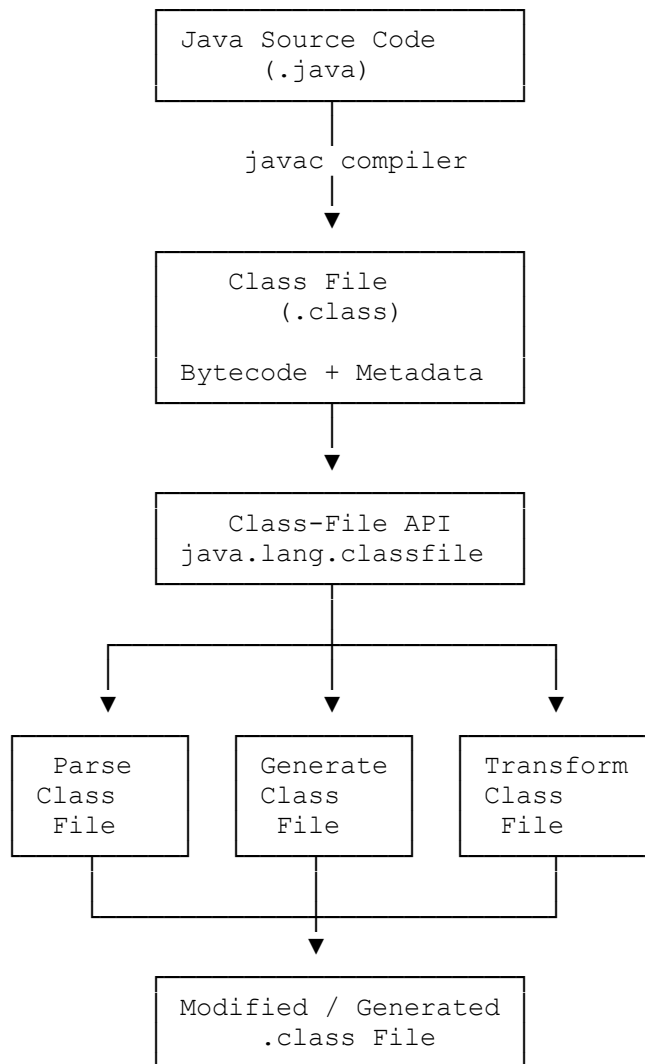
The Class-File API provides a modern JDK-supported alternative based on the `java.lang.classfile` package.

Main Capabilities

Class-File API

- Parse .class files
 - Inspect class structure
 - Generate class files
 - Transform class files
-

Flowchart



Java Code Example — Reading a Class File

Because this is a **Java 22 preview feature**, compile and run with preview enabled.

```
import java.lang.classfile.ClassFile;
import java.nio.file.Files;
import java.nio.file.Path;

public class ClassFileAPI {

    public static void main(String[] args) throws Exception {

        // Path to a compiled .class file
        Path classFilePath =
```

```

        Path.of("ClassFileAPI.class");

    // Read class-file bytes
    byte[] bytes =
        Files.readAllBytes(classFilePath);

    // Create ClassFile API instance
    ClassFile classFile =
        ClassFile.of();

    // Parse the class file
    var classModel =
        classFile.parse(bytes);

    // Display class information
    System.out.println(
        "Class: " + classModel.thisClass()
    );

    System.out.println(
        "Superclass: " + classModel.superclass()
    );

    System.out.println(
        "Class file parsed successfully."
    );
}
}

```

Example Output

```

Class: ClassFileAPI
Superclass: java/lang/Object
Class file parsed successfully.

```

Generating a Class File

The API can also generate a class file programmatically.

```

import java.lang.classfile.ClassFile;
import java.lang.constant.ClassDesc;
import java.nio.file.Files;
import java.nio.file.Path;

public class ClassFileAPI {

    public static void main(String[] args) throws Exception {

        ClassFile classFile =
            ClassFile.of();
    }
}

```

```

        ClassDesc classDesc =
            ClassDesc.of("GeneratedClass");

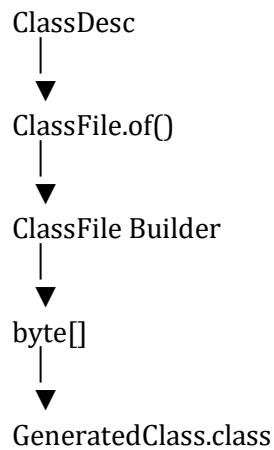
        byte[] bytes =
            classFile.build(
                classDesc,
                builder -> {
                    // Class-file elements can be added here.
                }
            );

        Files.write(
            Path.of("GeneratedClass.class"),
            bytes
        );

        System.out.println(
            "GeneratedClass.class created."
        );
    }
}

```

The important concept is:



Transforming a Class File

One of the most important use cases is **bytecode transformation**.

Conceptually:

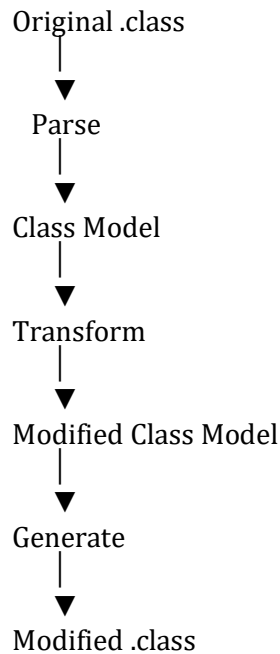
```
byte[] originalClass = ...;
```

```
ClassFile classFile = ClassFile.of();
```

```
byte[] modifiedClass =
```

```
classFile.transform(  
    classFile.parse(originalClass),  
    ...  
);
```

The transformation process is:



This is useful for technologies such as:

- bytecode instrumentation
- profiling
- monitoring
- framework generation
- dynamic proxies
- code transformation
- JVM tooling

Important Classes and Interfaces

API	Purpose
ClassFile	Main entry point for parsing, generating, and transforming class files
ClassModel	Represents the contents of a class file
ClassBuilder	Builds a class file

MethodModel	Represents a method
MethodBuilder	Builds a method
CodeModel	Represents bytecode instructions
CodeBuilder	Builds bytecode
ClassDesc	Describes a class using the constant API

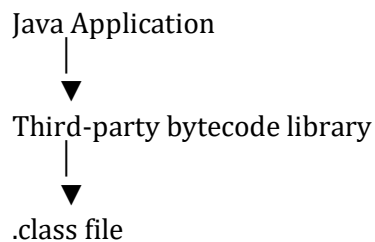
The package is:

java.lang.classfile

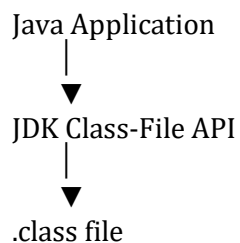
Why Was Class-File API Introduced?

Before this API, developers who needed to manipulate bytecode frequently depended on external libraries.

Traditional Approach



Java 22 Approach



This gives the JDK a **standard API for class-file processing** and provides a foundation for JVM tooling and libraries that need to work directly with bytecode.

Class-File API vs Reflection

These APIs solve different problems:

Reflection	Class-File API
Works with loaded classes	Works with class-file structures
Inspects runtime objects/classes	Parses/builds .class files
Can invoke methods	Can manipulate bytecode
Runtime-oriented	Class-file/JVM-oriented

For example:

Reflection
↓
Loaded Class
↓
Runtime information

whereas:

Class-File API
↓
.class file
↓
Bytecode + class metadata

Key Point

The Class-File API in Java 22 is a preview API that provides a standard JDK mechanism for reading, generating, and transforming JVM class files, reducing the need for external bytecode-manipulation libraries.

Java 22 — String Templates (Second Preview)

Java Version: 22

Year: 2024

Feature: String Templates (Second Preview)

UniqueClassName: StringTemplates

Definition

String Templates were a **second-preview feature in Java 22**, specified by **JEP 459**.

They were designed to make it easier and safer to combine **literal text with expressions**.

A string template consists conceptually of:

Template + Embedded Expressions



String Processor



Resulting String

For example, the intended syntax was:

```
String name = "Anurag";  
int age = 30;
```

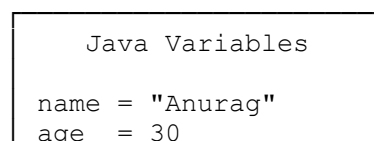
```
String message = STR."Name: \{name}, Age: \{age}";
```

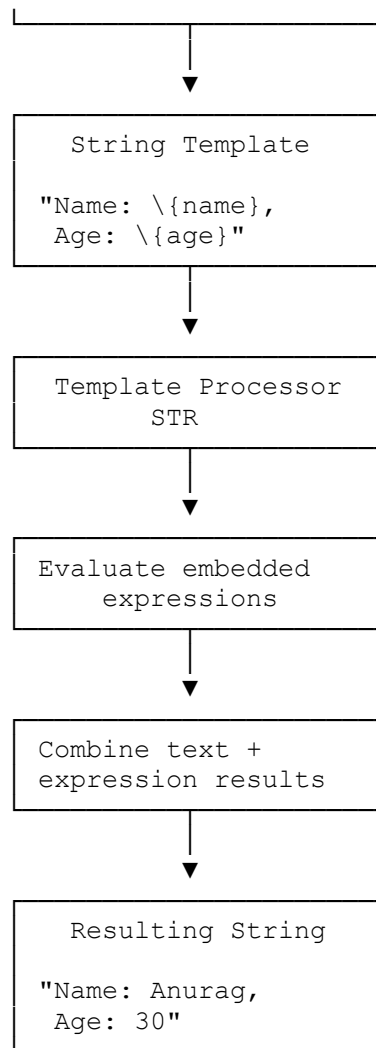
Here:

- STR is a **template processor**.
- "Name: \{name}, Age: \{age}" is the template.
- \{name} and \{age} are embedded expressions.

Important: String Templates were ultimately **withdrawn in JDK 23**, so this Java 22 example is specifically for the Java 22 preview feature and is not available as a normal permanent Java feature in current JDKs.

Flowchart





Java Code Example

Because this was a **preview feature in Java 22**, preview support was required.

```
public class StringTemplates {  
  
    public static void main(String[] args) {  
  
        String name = "Anurag";  
        int age = 30;  
  
        String message =  
            STR."Name: \#{name}, Age: \#{age}";  
  
        System.out.println(message);  
    }  
}
```

Output

Name: Anurag, Age: 30

Expression Inside a String Template

Expressions could be more than simple variables:

```
public class StringTemplates {  
  
    public static void main(String[] args) {  
  
        int a = 10;  
        int b = 20;  
  
        String result =  
            STR."\\{a} + \\{b} = \\{a + b}";  
  
        System.out.println(result);  
    }  
}
```

Output

10 + 20 = 30

The expression:

\\{a + b}

is evaluated first and its result is inserted into the template.

Multi-Line String Template

String templates could also be combined with text blocks:

```
public class StringTemplates {  
  
    public static void main(String[] args) {  
  
        String name = "Anurag";  
        int score = 95;  
  
        String report = STR."""  
            Student: \\{name}  
            Score:    \\{score}  
        """  
    }  
}
```

```
        Status:  \{score >= 50 ? "PASS" : "FAIL"}
        """;
    }
    System.out.println(report);
}
```

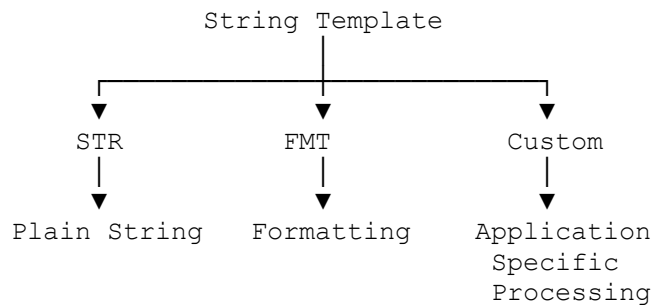
Output

Student : Anurag
Score : 95
Status : PASS

Custom Template Processors

One of the interesting aspects of String Templates was that STR was **not the only possible processor**.

Conceptually:



For example:

```
String name = "Anurag";
```

```
String result = STR."Hello, \{name}";
```

STR performs ordinary string interpolation.

A custom processor could theoretically perform additional processing, such as validation, escaping, formatting, or constructing another type.

Traditional Concatenation vs String Templates

Traditional

```
String name = "Anurag";  
int age = 30;
```

```
String message =  
    "Name: " + name + ", Age: " + age;
```

String Templates — Java 22 Preview

```
String name = "Anurag";  
int age = 30;
```

```
String message =  
    STR."Name: \{name}, Age: \{age}";
```

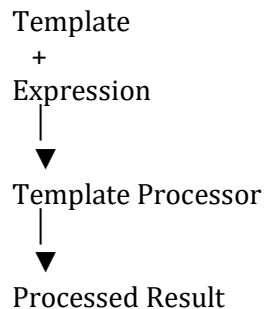
The template version makes the relationship between the text and embedded expressions easier to see.

Why String Templates Were Proposed

The feature was designed to provide:

- **Readable string interpolation**
- **Embedded Java expressions**
- **Custom template processors**
- **Better support for formatted strings**
- **Potentially safer processing of structured data**

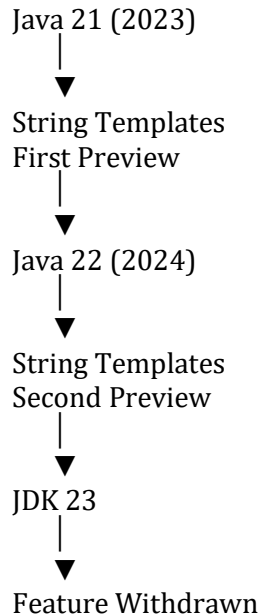
The important architectural idea was:



rather than simply performing string concatenation.

Important Version Note

The Java release history is important here:



So, when documenting **Java 22 features**, String Templates (Second Preview) is correct. However, it should **not** be described as a final Java 22 feature.

Key Point

String Templates in Java 22 were a second-preview feature that introduced STR."..."-style templates with embedded `\{...}` expressions and template processors, but the feature was later withdrawn rather than becoming a permanent Java language feature.

Java 22 — Vector API (Seventh Incubator)

Java Version: 22

Year: 2024

Feature: Vector API (Seventh Incubator)

UniqueClassName: VectorAPI

Definition

The **Vector API (Seventh Incubator)** was an **incubator feature in Java 22**, specified by **JEP 460**.

It provides an API for expressing **vector computations** that can be compiled by the JVM into efficient **SIMD (Single Instruction, Multiple Data)** instructions supported by modern CPUs.

Instead of processing values one at a time:

Scalar:

$A[0] + B[0] \rightarrow C[0]$

$A[1] + B[1] \rightarrow C[1]$

$A[2] + B[2] \rightarrow C[2]$

$A[3] + B[3] \rightarrow C[3]$

the Vector API can process multiple values simultaneously:

Vector A: [A0 A1 A2 A3]

+

Vector B: [B0 B1 B2 B3]

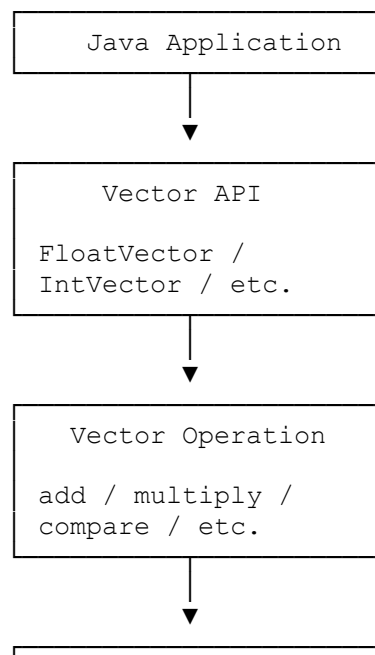
↓

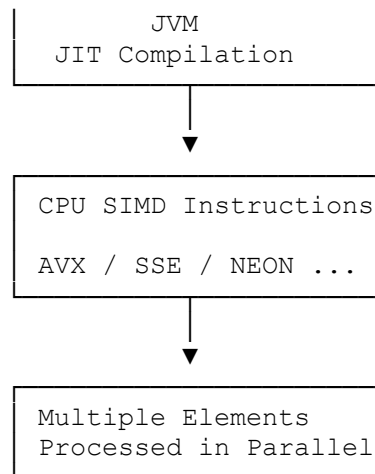
Vector C: [C0 C1 C2 C3]

This can improve performance for computationally intensive workloads such as:

- scientific computing
- machine learning
- image processing
- signal processing
- numerical algorithms
- cryptography
- data analytics

Flowchart





Java Code Example

A simple vector addition example:

```
import jdk.incubator.vector.IntVector;
import jdk.incubator.vector.VectorSpecies;

public class VectorAPI {

    public static void main(String[] args) {

        int[] a = {1, 2, 3, 4, 5, 6, 7, 8};
        int[] b = {10, 20, 30, 40, 50, 60, 70, 80};
        int[] result = new int[a.length];

        // Preferred vector species for the current CPU
        VectorSpecies<Integer> species =
            IntVector.SPECIES_PREFERRED;

        int i = 0;

        // Process elements in vector-sized chunks
        for (; i < species.loopBound(a.length);
            i += species.length()) {

            IntVector va =
                IntVector.fromArray(species, a, i);

            IntVector vb =
                IntVector.fromArray(species, b, i);

            IntVector vc =
                va.add(vb);

            vc.intoArray(result, i);
        }
    }
}
```

```

        // Process remaining elements
        for (; i < a.length; i++) {
            result[i] = a[i] + b[i];
        }

        for (int value : result) {
            System.out.print(value + " ");
        }
    }
}

```

Output

11 22 33 44 55 66 77 88

How the Example Works

1. Define Input Arrays

```

int[] a = {1, 2, 3, 4, 5, 6, 7, 8};
int[] b = {10, 20, 30, 40, 50, 60, 70, 80};

```

We want:

$A + B$

which produces:

[11, 22, 33, 44, 55, 66, 77, 88]

2. Select the Preferred Vector Size

```

VectorSpecies<Integer> species =
    IntVector.SPECIES_PREFERRED;

```

`SPECIES_PREFERRED` allows the JVM to select a vector shape appropriate for the underlying hardware.

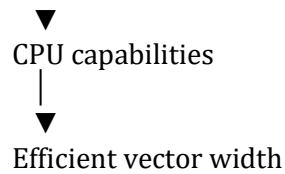
Conceptually:

Java Code



Preferred Vector Species





3. Load Data into Vectors

```
IntVector va =  
    IntVector.fromArray(species, a, i);
```

```
IntVector vb =  
    IntVector.fromArray(species, b, i);
```

Instead of loading one integer at a time, several integers can be represented by a vector.

For example:

```
va = [1 2 3 4]  
vb = [10 20 30 40]
```

4. Perform Vector Addition

```
IntVector vc = va.add(vb);
```

Conceptually:

```
[1  2  3  4]  
  +  
[10 20 30 40]  
  |  
  ▼  
[11 22 33 44]
```

One vector operation represents the computation over multiple elements.

5. Store the Result

```
vc.intoArray(result, i);
```

The vector contents are written back to the Java array.

Scalar vs Vector Processing

Traditional Scalar Processing

```
for (int i = 0; i < a.length; i++) {
    result[i] = a[i] + b[i];
}
```

Conceptually:

```
1 + 10
2 + 20
3 + 30
4 + 40
...
```

Vector Processing

```
IntVector va = IntVector.fromArray(species, a, i);
IntVector vb = IntVector.fromArray(species, b, i);
```

```
IntVector resultVector = va.add(vb);
```

Conceptually:

```
[1  2  3  4]
+
[10 20 30 40]
↓
[11 22 33 44]
```

The JVM can map suitable vector operations to hardware SIMD instructions.

Important Classes

Class/API	Purpose
IntVector	Vector operations on int values
FloatVector	Vector operations on float values
DoubleVector	Vector operations on double values
LongVector	Vector operations on long values
VectorSpecies	Describes vector shape/species
VectorMask	Represents a mask for conditional vector operations

VectorShuffle	Represents vector element rearrangements
VectorOperators	Provides vector operation definitions

The API is located in:

jdk.incubator.vector

Vector Mask Example

Vector masks allow operations to be performed conditionally.

```
import jdk.incubator.vector.*;

public class VectorAPI {

    public static void main(String[] args) {

        VectorSpecies<Integer> species =
            IntVector.SPECIES_PREFERRED;

        int[] values = {1, 2, 3, 4, 5, 6, 7, 8};

        IntVector vector =
            IntVector.fromArray(species, values, 0);

        VectorMask<Integer> mask =
            vector.compare(
                VectorOperators.GT,
                4
            );

        System.out.println(mask);
    }
}
```

Conceptually:

Vector:
[1 2 3 4 5 6 7 8]

Condition:
value > 4

Mask:
[F F F F T T T T]

Compilation in Java 22

Because the Vector API was an **incubator module**, it must be added when compiling and running:

```
javac --add-modules jdk.incubator.vector VectorAPI.java
```

Run:

```
java --add-modules jdk.incubator.vector VectorAPI
```

You may see an incubator-module warning from the JDK.

Benefits

Benefit	Description
SIMD processing	Processes multiple data elements together
High performance	Can utilize CPU vector instructions
Portable	Java code doesn't need architecture-specific assembly
JIT optimized	JVM can compile vector operations efficiently
Type-safe	Uses Java's type system rather than low-level native code
Useful for numerical workloads	Particularly valuable for arrays and mathematical operations

Key Point

The Java 22 Vector API (Seventh Incubator) provides a high-level, type-safe way to express SIMD computations so the JVM can potentially translate them into efficient hardware vector instructions and process multiple data elements in parallel.

Java 22 — Stream Gatherers (Preview)

Java Version: 22

Year: 2024

Feature: Stream Gatherers (Preview)

UniqueClassName: StreamGatherers

Definition

Stream Gatherers were introduced as a **preview feature in Java 22** through **JEP 461**.

A **gatherer** is a mechanism for defining **custom intermediate operations** for the Java Stream API.

The standard Stream API provides operations such as:

```
filter()
map()
flatMap()
distinct()
sorted()
limit()
```

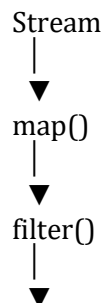
But some stream-processing problems are difficult or awkward to express using only these operations.

Stream Gatherers allow developers to create operations that can:

- maintain state while processing elements,
- process elements in groups,
- produce a different number of output elements,
- perform windowing,
- scan/reduce incrementally,
- implement custom stateful transformations.

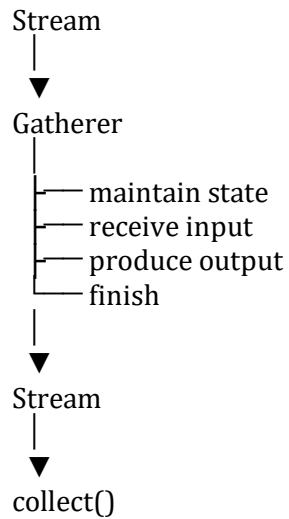
Basic Concept

Normally:



collect()

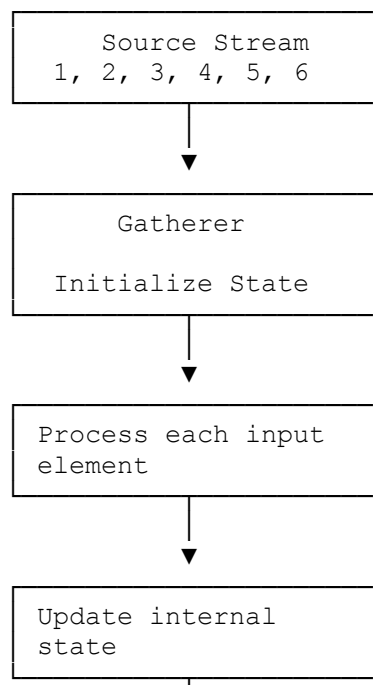
With a gatherer:

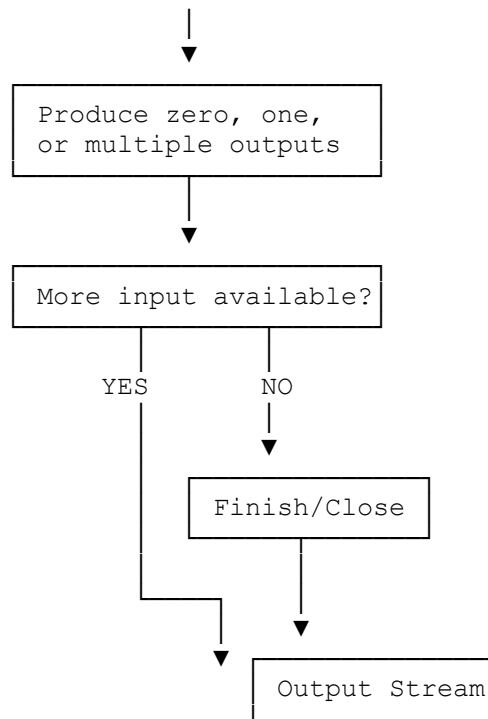


The key idea is:

A Gatherer controls how input stream elements are transformed into output elements while optionally maintaining state.

Flowchart





Java Code Example — Fixed-Size Windowing

One useful application of a gatherer is creating **fixed-size groups** from a stream.

For example:

Input:
1 2 3 4 5 6 7 8 9

Window size = 3

Output:
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

Java 22 Code

```

import java.util.List;
import java.util.stream.Gatherers;

public class StreamGatherers {

    public static void main(String[] args) {

        List<List<Integer>> windows =
            java.util.stream.IntStream.rangeClosed(1, 9)
                .boxed()
                .gather(Gatherers.windowFixed(3))
    }
}

```

```

        .toList();

    windows.forEach(System.out::println);
}

```

Output

```

[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

```

The important part is:

```

.gather(Gatherers.windowFixed(3))

```

It transforms the stream into fixed-size windows.

Sliding Window Example

Gatherers can also create **sliding windows**.

```

import java.util.stream.Gatherers;
import java.util.stream.IntStream;

public class StreamGatherers {

    public static void main(String[] args) {

        IntStream.rangeClosed(1, 5)
            .boxed()
            .gather(Gatherers.windowSliding(3))
            .forEach(System.out::println);
    }
}

```

Output

```

[1, 2, 3]
[2, 3, 4]
[3, 4, 5]

```

Notice the difference:

Fixed Window:

```

[1 2 3]
[4 5 6]

```

[7 8 9]

Sliding Window:

```
[1 2 3]
 [2 3 4]
  [3 4 5]
```

Gatherers.fold() Example

Gatherers can also maintain state while processing a stream.

For example, calculate a running sum:

```
import java.util.stream.Gatherers;
import java.util.stream.IntStream;

public class StreamGatherers {

    public static void main(String[] args) {

        IntStream.rangeClosed(1, 5)
            .boxed()
            .gather(
                Gatherers.fold(
                    () -> 0,
                    (sum, value) -> sum + value
                )
            )
            .forEach(System.out::println);
    }
}
```

Conceptually:

Input:

1 → 2 → 3 → 4 → 5

State:

```
0
├─ +1 → 1
├─ +2 → 3
├─ +3 → 6
├─ +4 → 10
└─ +5 → 15
```

Output:

15

Gatherers.scan() Example

scan() is useful when you want intermediate accumulated results.

```
import java.util.stream.Gatherers;
import java.util.stream.IntStream;

public class StreamGatherers {

    public static void main(String[] args) {

        IntStream.rangeClosed(1, 5)
            .boxed()
            .gather(
                Gatherers.scan(
                    () -> 0,
                    (sum, value) -> sum + value
                )
            )
            .forEach(System.out::println);
    }
}
```

Conceptually, the results are:

Input		Accumulated Output
1	→	1
2	→	3
3	→	6
4	→	10
5	→	15

Built-in Gatherers

Java 22 provided several useful built-in gatherers.

windowFixed()

Creates non-overlapping fixed-size windows.

```
.gather(Gatherers.windowFixed(3))
```

windowSliding()

Creates overlapping sliding windows.

```
.gather(Gatherers.windowSliding(3))
```

fold()

Performs a stateful fold and produces a final result.

```
.gather(  
  Gatherers.fold(  
    () -> 0,  
    (sum, value) -> sum + value  
  )  
)
```

scan()

Produces intermediate accumulated results.

```
.gather(  
  Gatherers.scan(  
    () -> 0,  
    (sum, value) -> sum + value  
  )  
)
```

mapConcurrent()

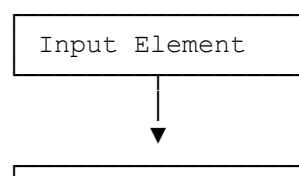
Can perform concurrent mapping with a configurable concurrency limit.

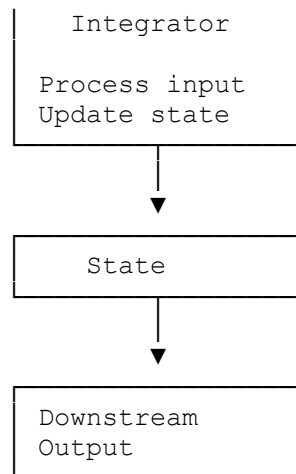
```
.gather(  
  Gatherers.mapConcurrent(  
    4,  
    value -> process(value)  
  )  
)
```

Custom Gatherer Concept

The real power of the API is the ability to define your own gatherer.

Conceptually:





A custom gatherer can define:

- initialization,
- integration of input elements,
- state,
- output,
- finalization,
- parallel-combination behavior.

Stream vs Gatherer

Stream Operation	Gatherer
map()	Custom stateful transformations
filter()	Complex stateful processing
flatMap()	Variable output patterns
reduce()	Incremental/stateful processing
collect()	Custom intermediate stream operations
Limited composition	Highly customizable intermediate operation

A gatherer is particularly useful when an operation doesn't fit naturally into a combination of existing map, filter, and flatMap operations.

Example: Processing Sensor Data

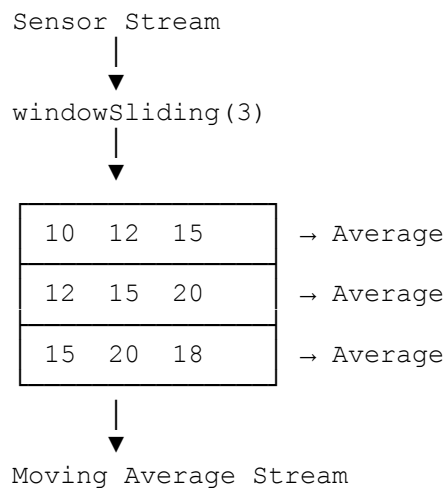
Imagine sensor readings:

10, 12, 15, 20, 18, 22, 25

A sliding-window gatherer can group readings:

```
[10, 12, 15]
[12, 15, 20]
[15, 20, 18]
[20, 18, 22]
[18, 22, 25]
```

You could then calculate a moving average for each window.



This is one example of the kind of stateful stream processing that Gatherers make easier.

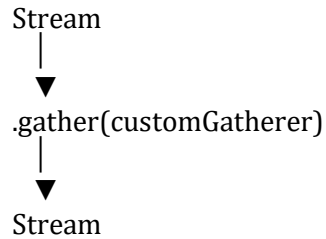
Why Stream Gatherers?

Before Gatherers, developers often had to use complicated combinations of:

```
map()
filter()
flatMap()
reduce()
collect()
```

or write custom collectors and external mutable state.

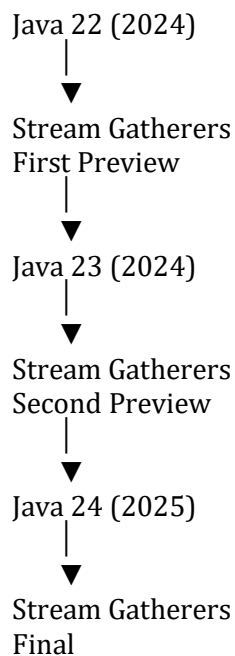
With Gatherers:



The custom intermediate operation becomes **composable with the normal Stream pipeline**.

Important Version Note

The feature progression was:



So for **Java 22**, the correct designation is **Stream Gatherers (Preview)**.

Key Point

Stream Gatherers in Java 22 provide a framework for creating custom intermediate Stream operations that can maintain state, process elements in groups, produce variable numbers of results, and perform sophisticated transformations that are difficult to express with traditional Stream operations alone.

Java 22 — Structured Concurrency (Second Preview)

Java Version: 22

Year: 2024

Feature: Structured Concurrency (Second Preview)

UniqueClassName: StructuredConcurrency

Definition

Structured Concurrency was a **second-preview feature in Java 22**, specified by **JEP 462**.

It provides an API for treating a group of related concurrent tasks as a **single unit of work**.

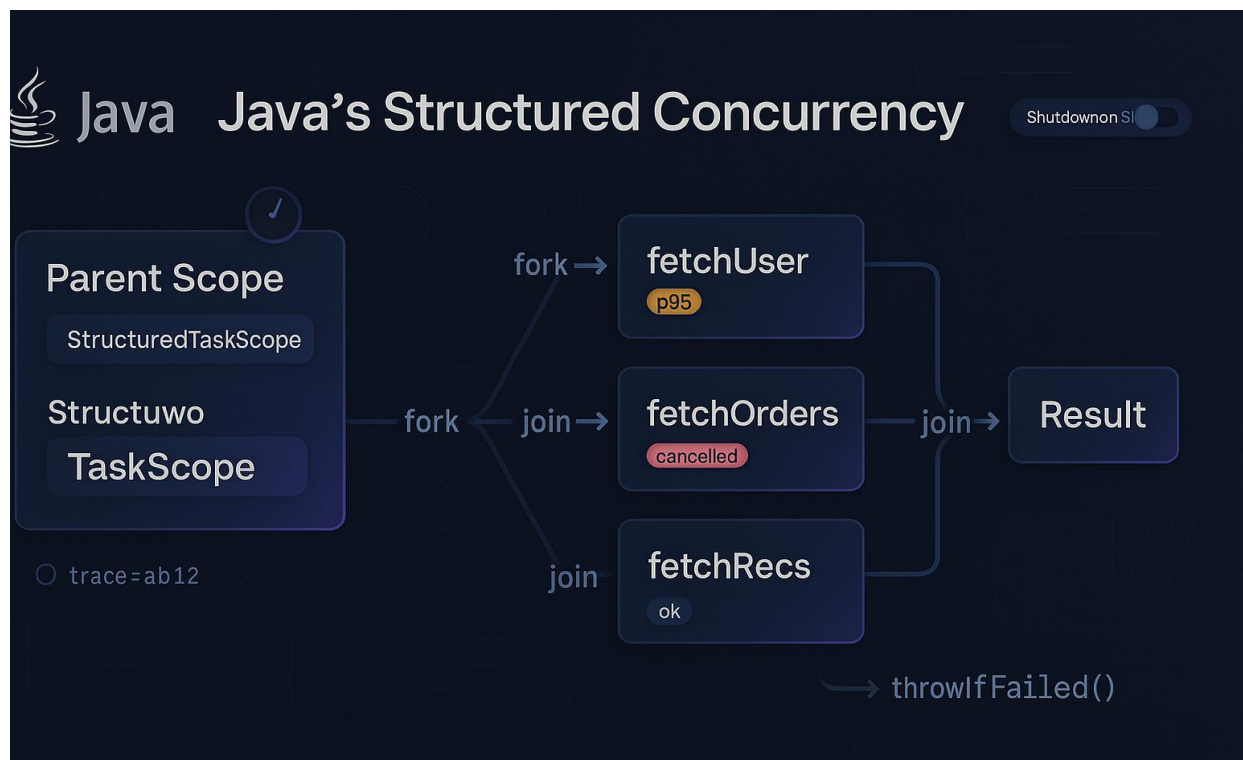
The central idea is:

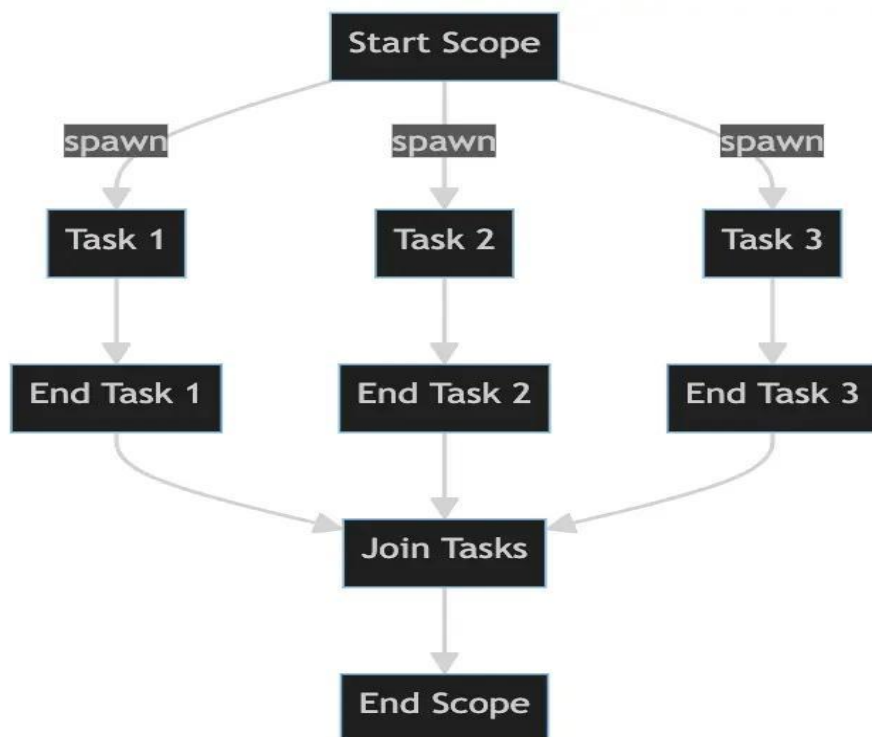
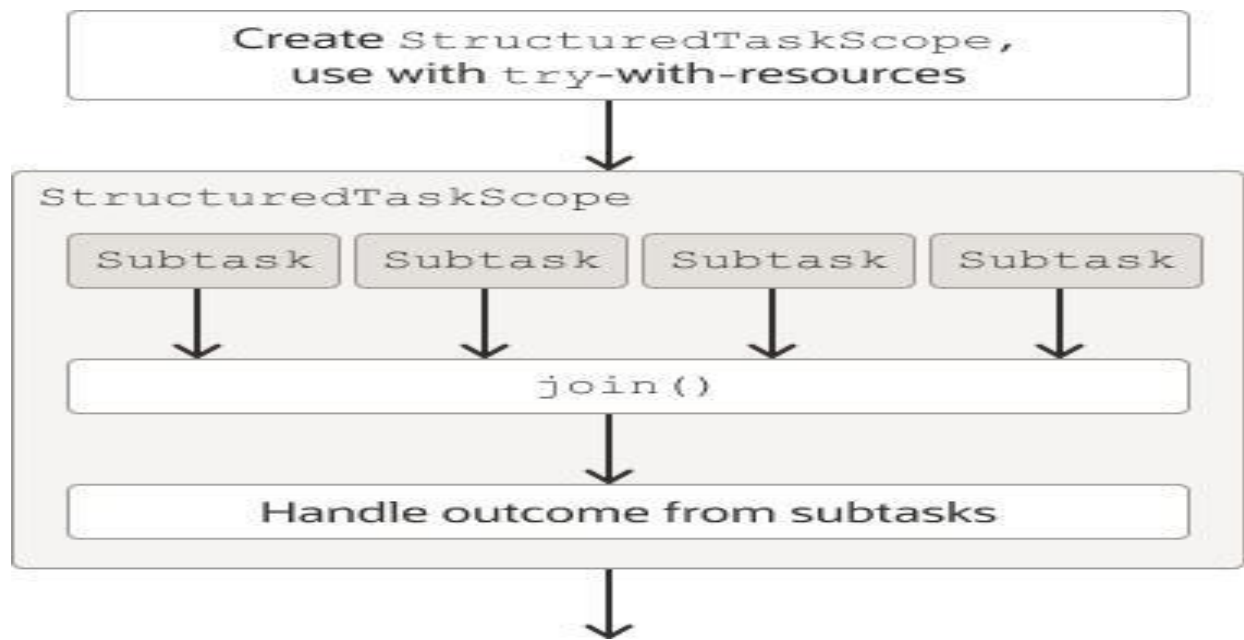
If a task forks subtasks, those subtasks should finish before the parent task finishes.

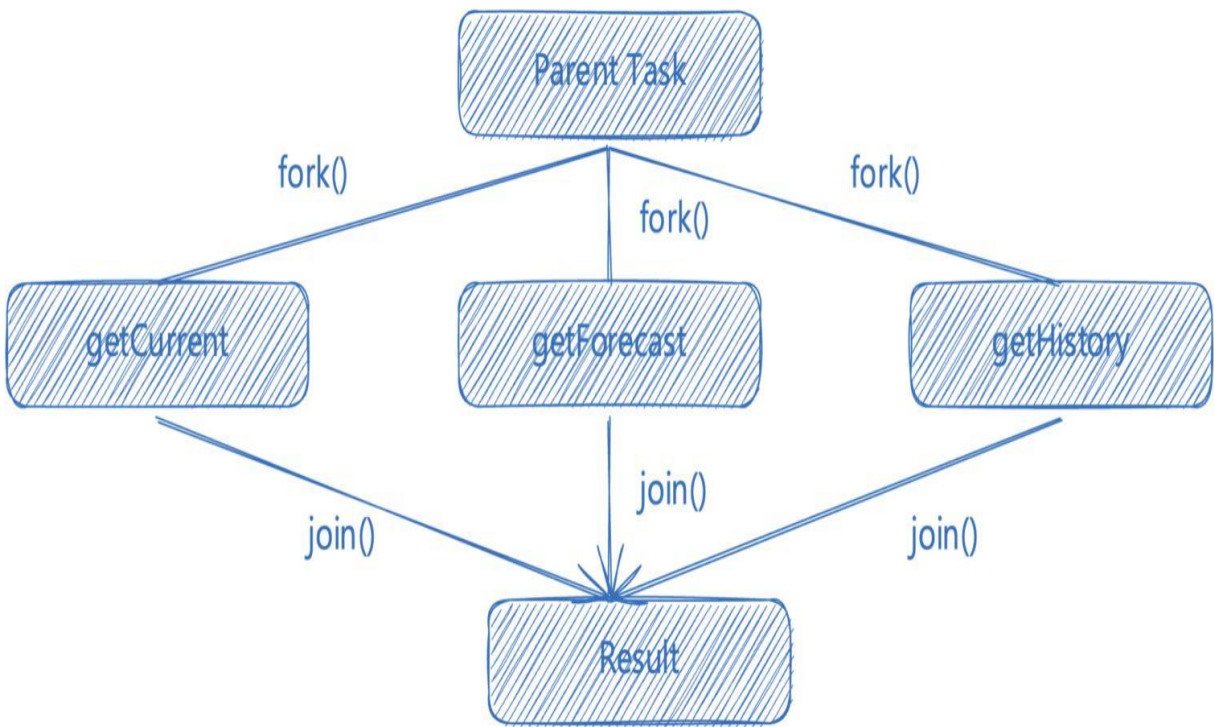
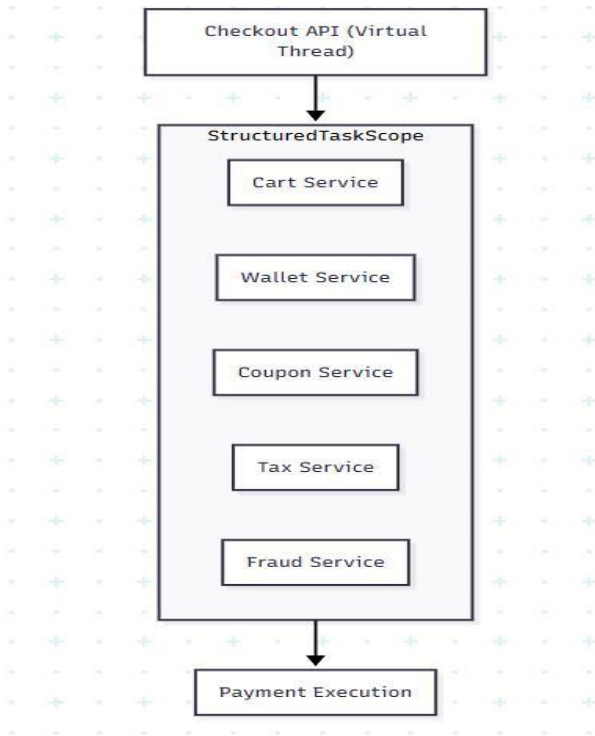
This makes concurrent code easier to reason about, handle errors in, and cancel.

The API is based primarily on:

StructuredTaskScope

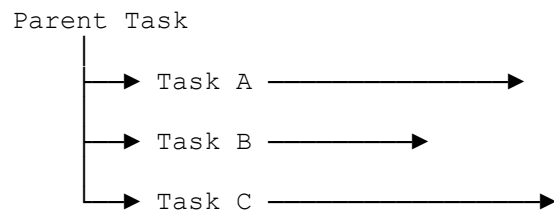






Traditional Concurrency vs Structured Concurrency

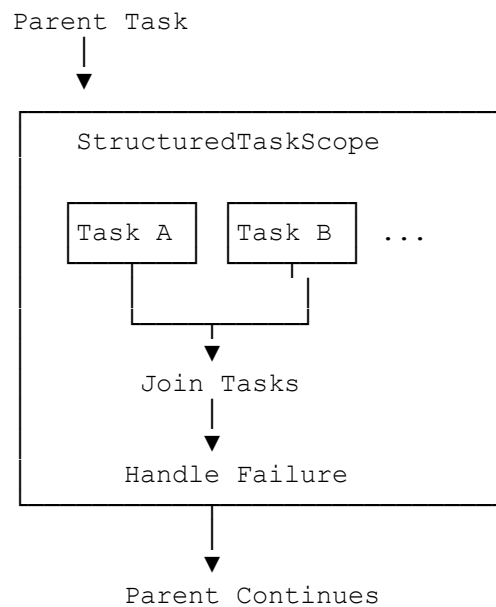
Traditional approach



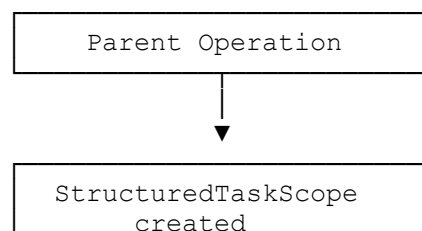
Parent may finish while
child tasks are still running.

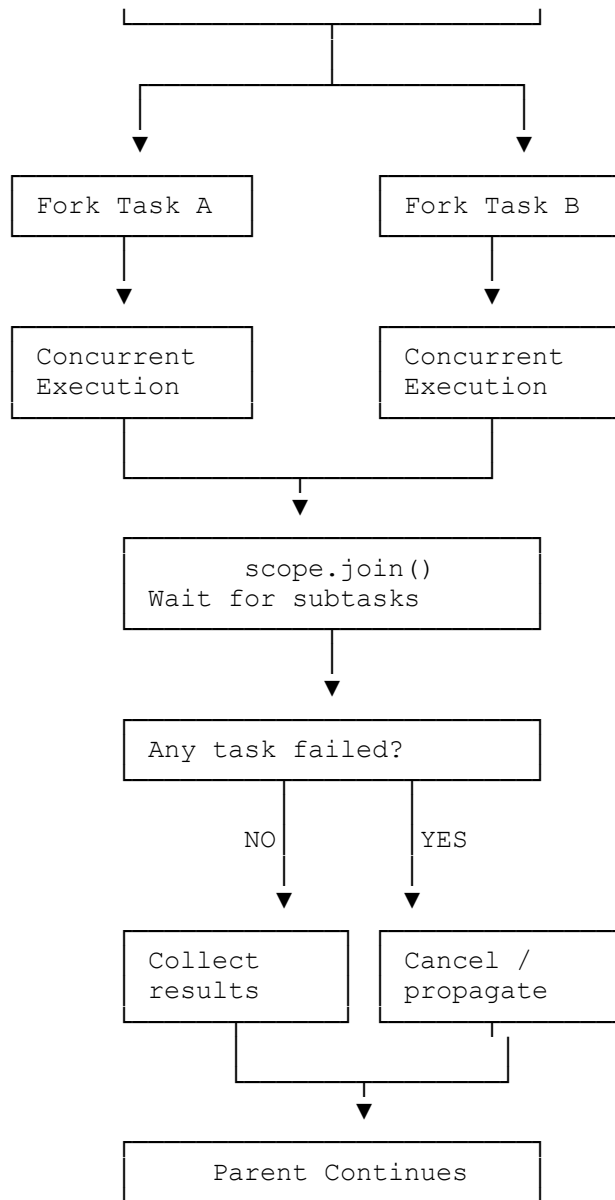
This can make failures, cancellation, and resource management difficult to coordinate.

Structured Concurrency



Flowchart





Java Code Example

Because this was a **preview feature in Java 22**, it requires preview compilation and execution.

The following example runs two independent tasks concurrently: fetching a user and fetching their order information.

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    static String getUser() throws InterruptedException {
        Thread.sleep(1000);
    }
}
```

```

        return "Anurag";
    }

    static String getOrders() throws InterruptedException {
        Thread.sleep(1500);
        return "5 orders";
    }

    public static void main(String[] args)
        throws InterruptedException {

        try (var scope =
            StructuredTaskScope.open()) {

            var userTask =
                scope.fork(StructuredConcurrency::getUser);

            var ordersTask =
                scope.fork(StructuredConcurrency::getOrders);

            // Wait for both subtasks
            scope.join();

            String user = userTask.get();
            String orders = ordersTask.get();

            System.out.println(
                "User: " + user
            );

            System.out.println(
                "Orders: " + orders
            );

        }
    }
}

```

Expected Output

```

User      : Anurag
Orders    : 5 orders

```

The two tasks can execute concurrently rather than sequentially:

Sequential:

```

getUser()      ———— 1 sec
                |
getOrders()    ———— 1.5 sec
                Total ~ 2.5 sec

```

Structured:

```
getUser()      _____ 1 sec
getOrders()    _____ 1.5 sec
                Total ≈ 1.5 sec
```

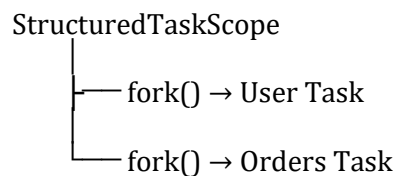
Actual elapsed time depends on scheduling and the environment.

Understanding fork()

```
var userTask =
  scope.fork(StructuredConcurrency::getUser);
```

fork() starts a concurrent subtask within the structured scope.

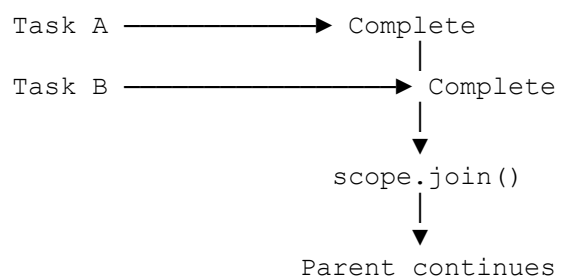
Conceptually:



Understanding join()

```
scope.join();
```

join() waits until the subtasks have completed or the scope is otherwise terminated.



This establishes the structured relationship between the parent and child tasks.

Handling Failure

One of the major benefits of structured concurrency is **coordinated failure handling**.

For example:

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    public static void main(String[] args)
        throws InterruptedException {

        try (var scope =
            StructuredTaskScope.open()) {

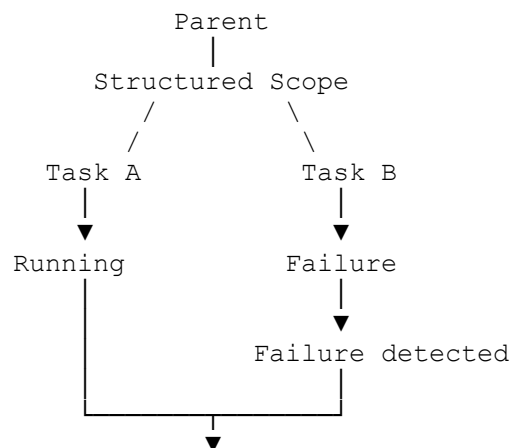
            var task1 = scope.fork(() -> {
                Thread.sleep(1000);
                return "Task 1 completed";
            });

            var task2 = scope.fork(() -> {
                throw new RuntimeException(
                    "Task 2 failed"
                );
            });

            scope.join();

            try {
                System.out.println(task1.get());
                System.out.println(task2.get());
            } catch (Exception e) {
                System.out.println(
                    "A subtask failed: "
                    + e.getMessage()
                );
            }
        }
    }
}
```

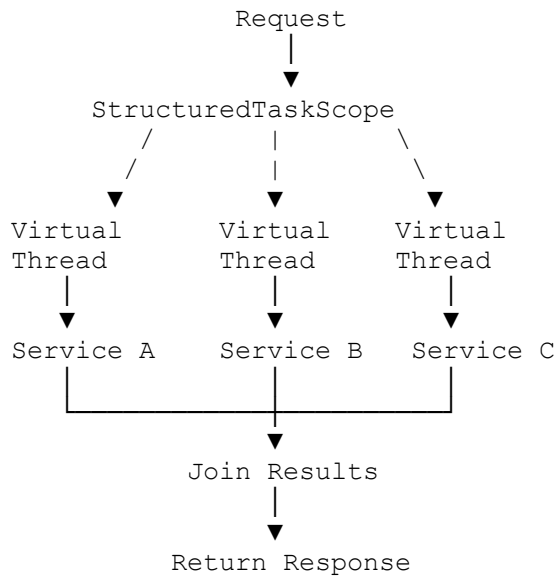
Conceptually:



Structured Concurrency + Virtual Threads

Structured Concurrency works particularly well with **virtual threads**.

A common architecture is:



This is useful for applications where one request needs data from multiple independent services.

Key Components

Component	Purpose
StructuredTaskScope	Defines the lifetime and structure of concurrent subtasks
fork()	Starts a subtask
join()	Waits for subtasks
Subtask	Represents the result/state of a forked task
get()	Retrieves a completed subtask's result
close()	Closes the structured scope

Why Structured Concurrency?

Without structured concurrency, code can become difficult to manage:

```
Parent
├── Thread A
├── Thread B
├── Thread C
└── Thread D
```

Who owns them?

Who cancels them?

What happens if one fails?

When are they finished?

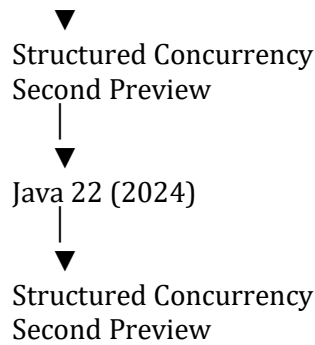
Structured concurrency provides a clear relationship:

```
Parent
┆
┆   ▼
┆ Scope
┆ ├── Child A
┆ ├── Child B
┆ └── Child C
┆   │
┆   ▼
┆ Join
┆   │
┆   ▼
┆ Handle results/errors
┆   │
┆   ▼
┆ Scope closes
```

The lifetime of the child tasks is bounded by the lifetime of the structured scope.

Java 22 Version History

```
Java 19 (2022)
┆
┆   ▼
┆ Structured Concurrency
┆ First Preview
┆   │
┆   ▼
┆ Java 21 (2023)
┆   │
```



Correction to note: JEP 462 in Java 22 was indeed the **second preview**, following JEP 453 in Java 21. The feature was subsequently refined in later releases.

Key Point

Structured Concurrency (Second Preview) in Java 22 provides a structured way to run related concurrent tasks as one unit, making task lifetimes, cancellation, failure handling, and result management easier to understand and control.

Java 22 — Implicitly Declared Classes and Instance Main Methods (Second Preview)

Java Version: 22

Year: 2024

Feature: Implicitly Declared Classes and Instance Main Methods (Second Preview)

UniqueClassName: ImplicitClassesInstanceMain

Definition

Implicitly Declared Classes and Instance Main Methods was a **second-preview feature in Java 22**, specified by **JEP 463**.

The feature makes Java easier for beginners by allowing a simple Java program to be written **without explicitly declaring a class** and without requiring the traditional:

```
public static void main(String[] args)
```

Instead, Java can implicitly create the class around the source file, and the main method can be an **instance method**.

Traditional Java

```
public class HelloWorld {
```

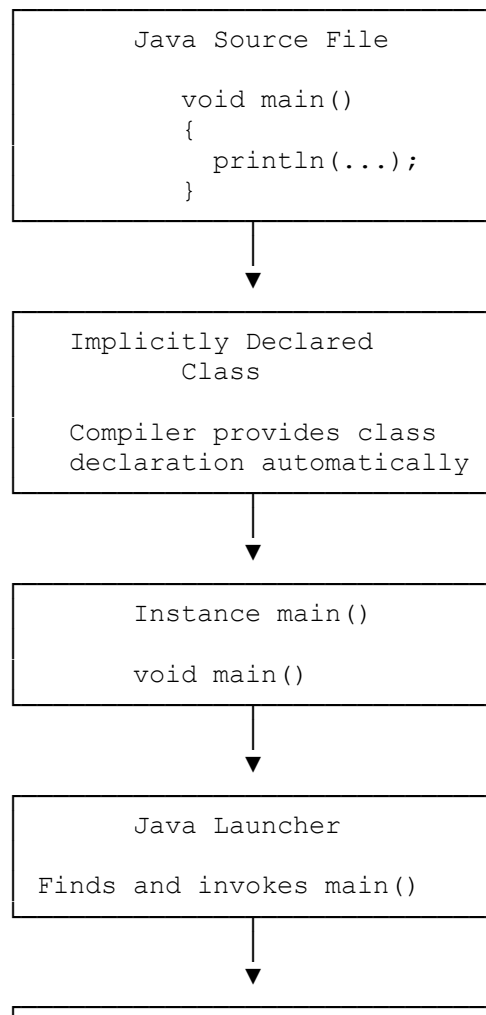
```
public static void main(String[] args) {  
    System.out.println("Hello, Java!");  
}  
}
```

Java 22 Preview

```
void main() {  
    System.out.println("Hello, Java!");  
}
```

The Java compiler/runtime conceptually treats the source as belonging to an implicitly declared class.

Flowchart



Java Code Example

A minimal Java 22 preview program:

```
void main() {  
    System.out.println("Hello, Java 22!");  
}
```

There is **no explicit**:

```
class MyClass {
```

and no:

```
public static void main(String[] args)
```

Output

Hello, Java 22!

Example with Variables and Methods

The feature isn't limited to a single `println()` statement.

```
String getMessage() {  
    return "Welcome to Java 22";  
}  
  
void main() {  
  
    String message = getMessage();  
  
    System.out.println(message);  
}
```

Conceptually, Java treats the source as if it were enclosed in an implicitly declared class:

```
class ImplicitClassesInstanceMain {  
  
    String getMessage() {  
        return "Welcome to Java 22";  
    }  
}
```

```
void main() {  
    String message = getMessage();  
    System.out.println(message);  
}  
}
```

The developer doesn't need to write that surrounding class declaration.

Instance main() Method

The important difference is:

Traditional

```
public static void main(String[] args)
```

Java 22 Preview

```
void main()
```

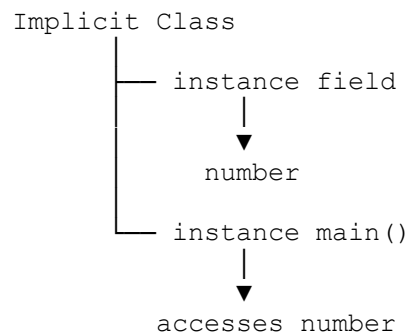
The new main() can be an **instance method**, so it can directly access instance members.

For example:

```
int number = 100;
```

```
void main() {  
    System.out.println(number);  
}
```

Conceptually:



Why Is This Useful?

Traditional Java has a significant amount of boilerplate for a very small program:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello World");  
  
    }  
}
```

The preview feature allows:

```
void main() {  
    System.out.println("Hello World");  
}
```

This makes Java programs more approachable for:

- beginners
- introductory programming courses
- small experiments
- simple scripts
- teaching basic Java syntax

Example — Simple Calculator

```
int add(int a, int b) {  
    return a + b;  
}  
  
void main() {  
  
    int x = 10;  
    int y = 20;  
  
    int result = add(x, y);  
  
    System.out.println("Result = " + result);  
}
```

Output

Result = 30

The source focuses directly on the programming concepts rather than class and main boilerplate.

Example — Command-Line Arguments

An instance main can also receive command-line arguments.

```
void main(String[] args) {  
  
    System.out.println("Number of arguments: "  
        + args.length);  
  
    for (String arg : args) {  
        System.out.println(arg);  
    }  
}
```

So the feature supports simplified entry points while retaining the ability to work with command-line arguments.

Main Method Forms

The feature expanded the forms of main methods that can be used as program entry points.

Conceptually:

```
main()  
├── void main()  
├── void main(String[] args)  
└── static variants
```

The key change is that main **doesn't have to be static**.

Traditional vs Java 22 Preview

Traditional Java	Java 22 Preview
Explicit class required	Class can be implicit
public static void main(...)	Instance void main() possible

More boilerplate	Less boilerplate
Class declaration visible	Class declaration can be omitted
Beginner sees more infrastructure	Beginner can focus on code

Traditional

```
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

Java 22

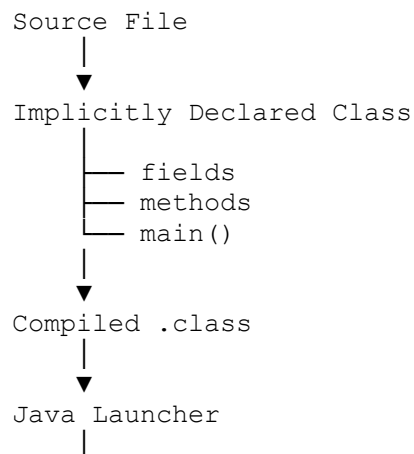
```
void main() {
    System.out.println("Hello");
}
```

How It Works Internally

The developer writes:

```
void main() {
    ...
}
```

The Java compiler conceptually associates the top-level declarations with an **implicitly declared class**:



▼
main()

The implicit class is still a real class from the JVM's perspective; the feature primarily removes the need for the programmer to write the surrounding declaration.

Compilation in Java 22

Because this was a **preview feature**, Java 22 requires preview options.

For example:

```
javac --enable-preview --release 22 HelloWorld.java
```

Run it with:

```
java --enable-preview HelloWorld
```

Java Version History

The feature progressed as follows:

```
Java 21 — 2023
    |
    ▼
Implicitly Declared Classes and
Instance Main Methods
First Preview
    |
    ▼
Java 22 — 2024
    |
    ▼
Second Preview
    |
    ▼
Further refinement in later releases
```

Important Note

This is a **preview language feature**, not a permanent Java 22 final feature. Its purpose was primarily to make Java's entry-level programming experience simpler while retaining the full Java language and JVM model underneath.

Key Point

Implicitly Declared Classes and Instance Main Methods (Second Preview) in Java 22 allow simple Java programs to omit the explicit class declaration and use an instance `main()` method, significantly reducing boilerplate and making Java easier to learn and use for small programs.

Java 22 — Scoped Values (Second Preview)

Java Version: 22

Year: 2024

Feature: Scoped Values (Second Preview)

UniqueClassName: ScopedValues

Definition

Scoped Values were a **second-preview feature in Java 22**, specified by **JEP 464**.

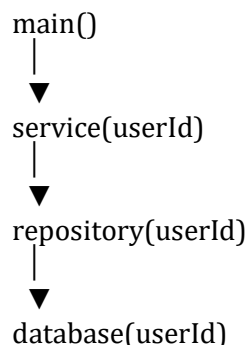
A **Scoped Value** provides a way to share an **immutable value** with methods and tasks executed within a particular scope.

It is especially useful for:

- passing context through deeply nested method calls,
- concurrent applications,
- virtual threads,
- structured concurrency,
- request-scoped data,
- security/context information.

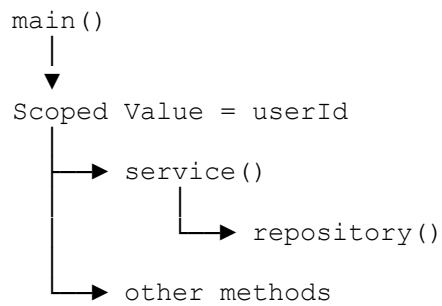
Unlike a normal parameter, the value doesn't need to be explicitly passed through every method.

Traditional Parameter Passing

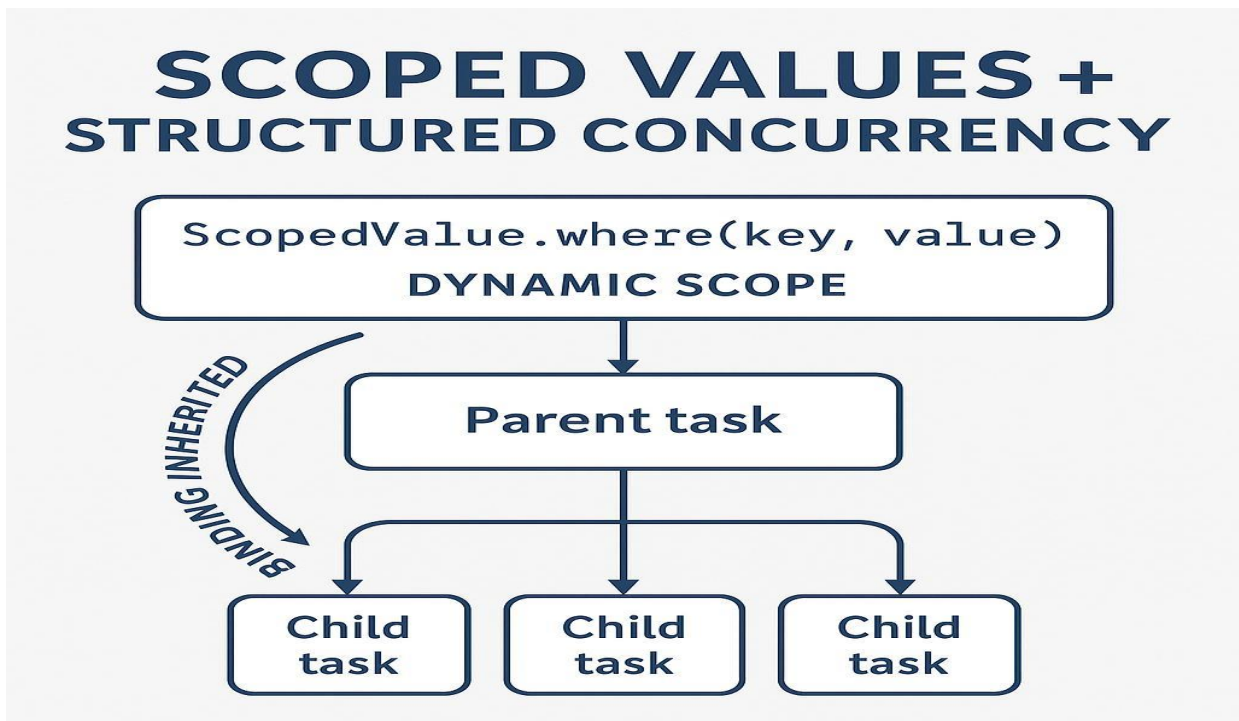


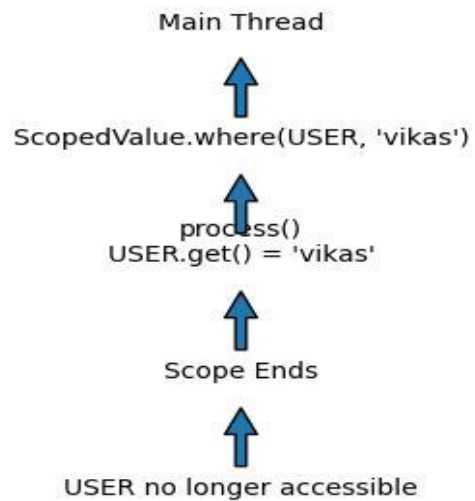
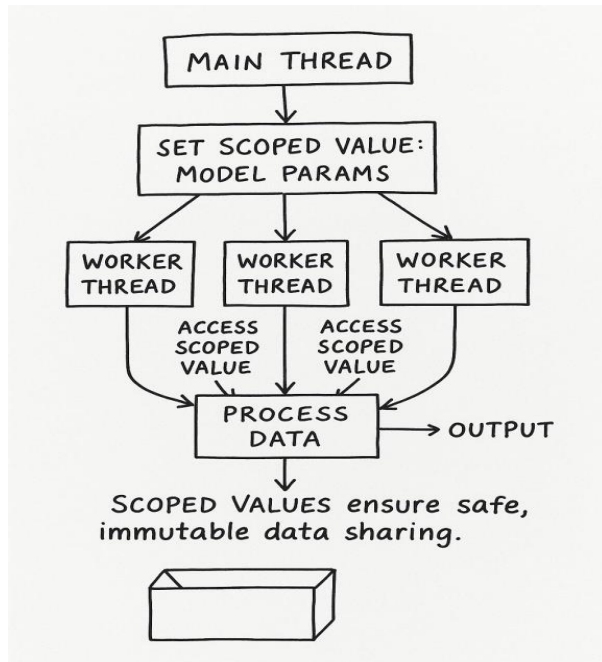
The value must be passed through each method.

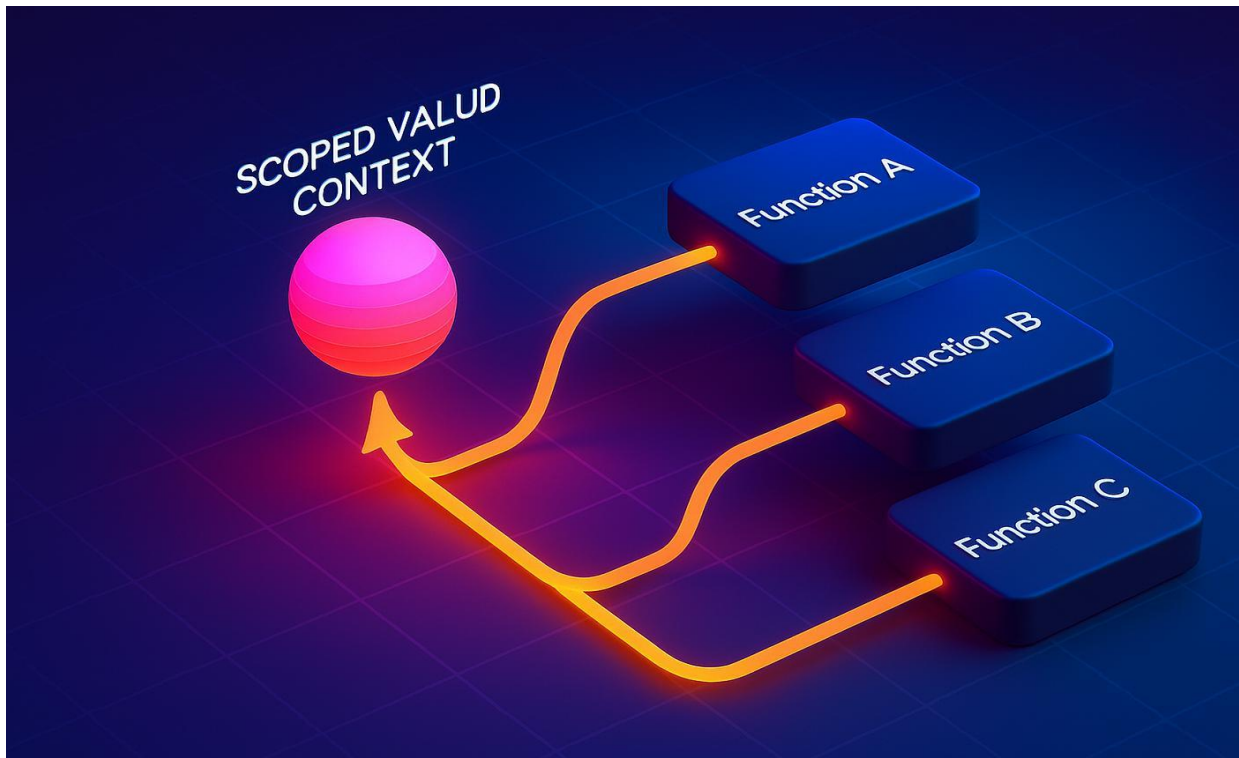
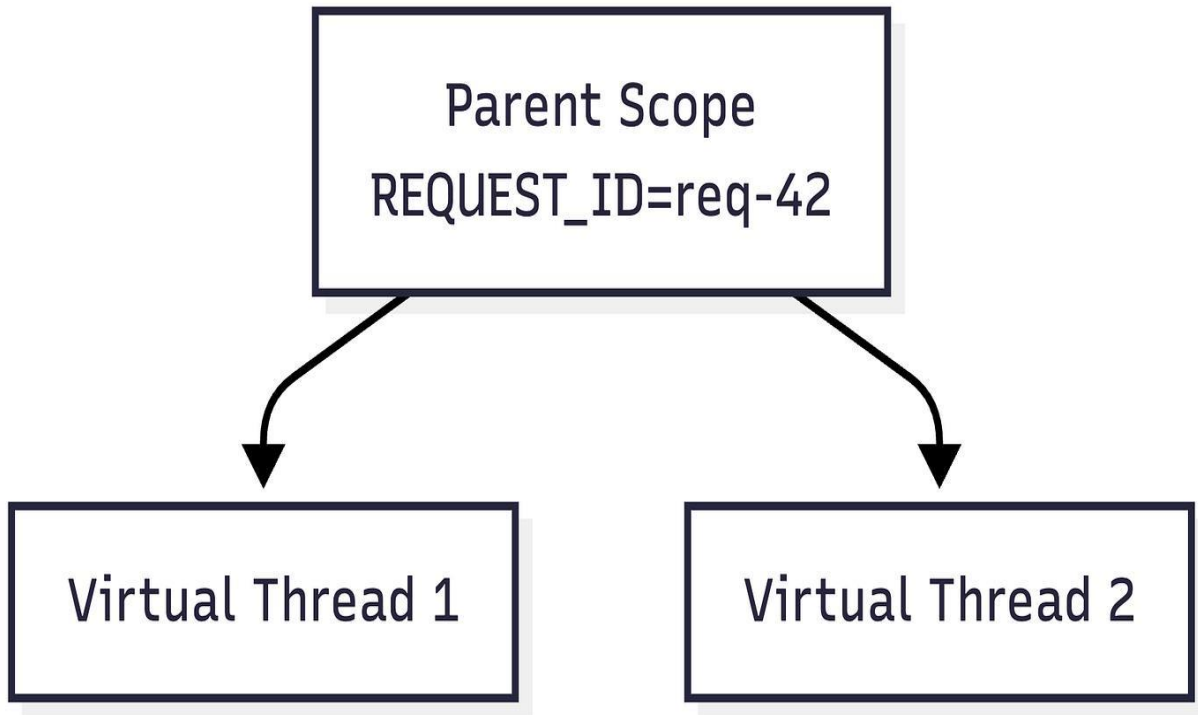
Scoped Value



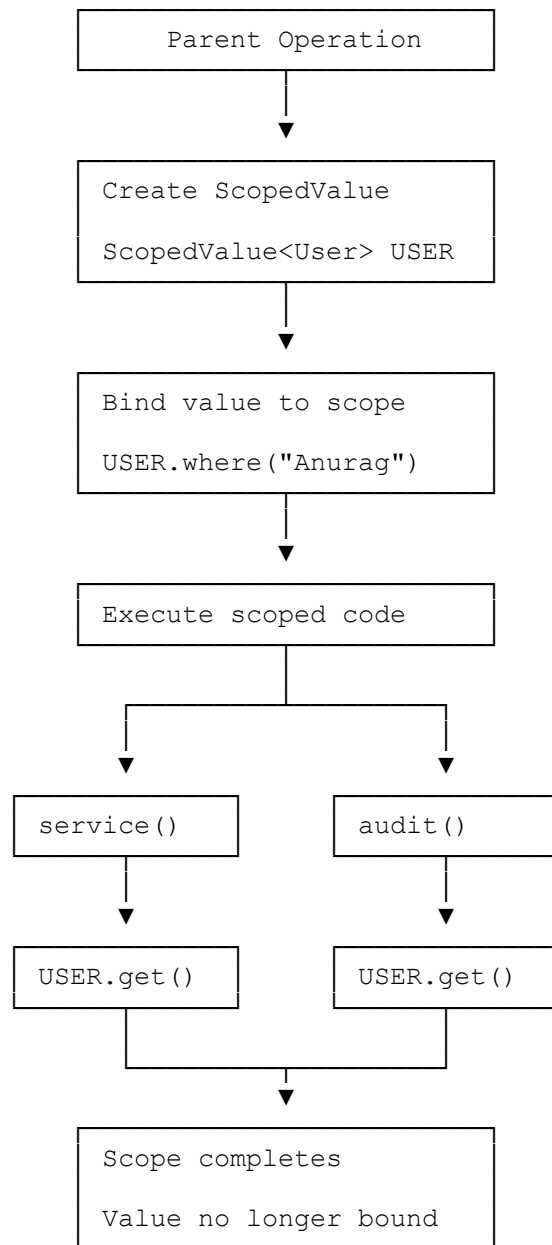
All methods can access the value while they execute inside the scope.







Flowchart



Java Code Example

Java 22 uses `ScopedValue` from `java.lang`:

```
import java.lang.ScopedValue;  
  
public class ScopedValues {  
    private static final ScopedValue<String> USER =
```

```

        ScopedValue.newInstance();

static void service() {
    System.out.println(
        "Service user: " + USER.get()
    );

    repository();
}

static void repository() {
    System.out.println(
        "Repository user: " + USER.get()
    );
}

public static void main(String[] args) {

    ScopedValue.where(USER, "Anurag")
        .run(() -> {
            service();
        });
}
}

```

Output

```

Service user: Anurag
Repository user: Anurag

```

Notice that `service()` and `repository()` don't receive the user as a parameter:

```

service();
repository();

```

Yet both methods can access:

```

USER.get()

```

because the value is bound to the current scope.

How It Works

1. Create a Scoped Value

```

private static final ScopedValue<String> USER =
    ScopedValue.newInstance();

```

This creates a key representing the scoped value.

Think of it as:

USER
|
▼
Scoped Value Key

The key itself does not contain the user value until it is bound to a scope.

2. Bind a Value

```
ScopedValue.where(USER, "Anurag")
```

This creates a new scope in which:

USER → "Anurag"

3. Execute Code Within the Scope

```
.run() -> {  
    service();  
});
```

Everything executed within that scope can access the binding.

```
USER = "Anurag"  
  |  
  ▼  
run()  
├── service()  
│   └── repository()  
└── other methods
```

4. Read the Value

```
USER.get()
```

The current binding is retrieved.

For example:

```
static void service() {
```

```
String user = USER.get();  
  
System.out.println(user);  
}
```

Scoped Value vs Method Parameter

Method Parameters

```
void service(String user) {  
    repository(user);  
}  
  
void repository(String user) {  
    System.out.println(user);  
}
```

The value must be explicitly passed:

```
main  
|  
▼  
service(user)  
|  
▼  
repository(user)
```

Scoped Value

```
void service() {  
    repository();  
}  
  
void repository() {  
    System.out.println(USER.get());  
}
```

The value flows through the execution context:

```
Scoped Value  
|  
▼  
service()  
|  
▼  
repository()
```

|
▼
USER.get()

Scoped Values with Nested Scopes

A scoped value can be rebound in a nested scope.

```
import java.lang.ScopedValue;

public class ScopedValues {

    private static final ScopedValue<String> USER =
        ScopedValue.newInstance();

    public static void main(String[] args) {

        ScopedValue.where(USER, "Anurag")
            .run(() -> {

                System.out.println(
                    "Outer: " + USER.get()
                );

                ScopedValue.where(USER, "Rahul")
                    .run(() -> {

                        System.out.println(
                            "Inner: " + USER.get()
                        );
                    });

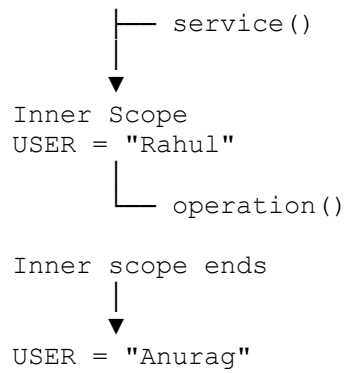
                System.out.println(
                    "Outer again: " + USER.get()
                );
            });
    }
}
```

Output

```
Outer          : Anurag
Inner          : Rahul
Outer again    : Anurag
```

This demonstrates **dynamic scoping**:

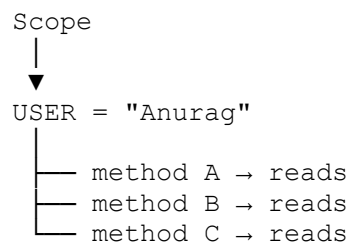
```
Outer Scope
USER = "Anurag"
|
```



Important Characteristics

1. Immutable

A `ScopedValue` is designed for sharing values rather than repeatedly modifying shared state.



Methods can read the value, but they don't modify the binding itself.

2. Scoped Lifetime

The value exists only within its binding scope.

```

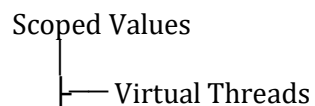
ScopedValue.where(USER, "Anurag")
    .run(() -> {
        // USER is available here
    });
  
```

```

// USER is no longer bound here
  
```

3. Works Well with Concurrency

Scoped values were designed with modern Java concurrency in mind, particularly:

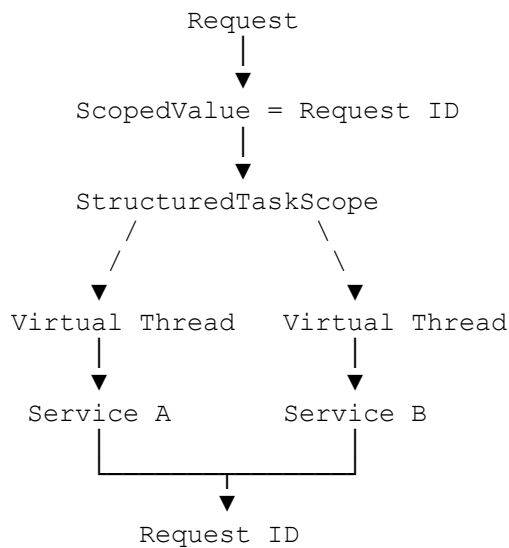


Structured Concurrency

This makes them useful for carrying request context through concurrent operations.

Example with Structured Concurrency

Conceptually:



For example, both concurrent operations can access the same request context without passing it through every method argument.

Benefits

Benefit	Description
Less parameter passing	Context doesn't need to be threaded through every method
Immutable sharing	Values are intended to be safely shared within a scope
Clear lifetime	Value exists only for the defined scope
Concurrency-friendly	Designed with modern concurrent programming in mind

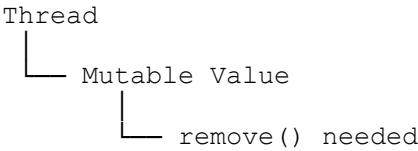
Virtual-thread friendly	Suitable for large numbers of lightweight tasks
Structured context	Works naturally with structured task execution

ThreadLocal vs ScopedValue

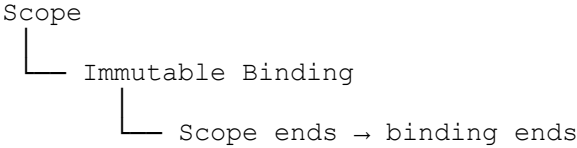
ThreadLocal	ScopedValue
Value is associated with a thread	Value is associated with a scope
Usually mutable	Designed for immutable bindings
Can remain until explicitly removed	Automatically bounded by scope
Historically useful for thread context	Designed for modern concurrency
Can be awkward with many virtual threads	Designed with virtual-thread usage in mind

Conceptually:

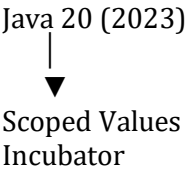
ThreadLocal:

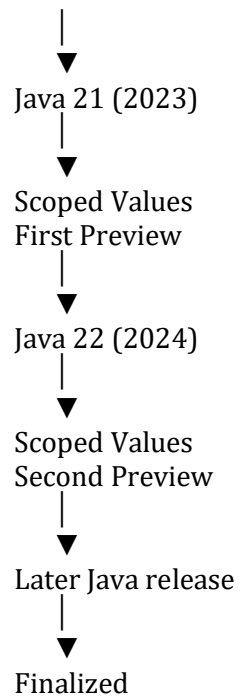


ScopedValue:



Java 22 Version History





Key Point

Scoped Values (Second Preview) in Java 22 provide a structured, immutable mechanism for sharing contextual data across method calls and concurrent tasks without explicitly passing that data through every method parameter.